

CPT102 Data Structures and Algorithms

数据结构和算法

内容基于 **LearningMall**的PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。



GITHUB

REPOSITORY



VISIT NOTES AUTHOR'S PAGE

CLICK ME

■ [Week 0 Lecture 0]

课程情况

Steven Guan	Room SD425	steven.guan@xjtlu.edu.cn
Kok Hoe WONG	Room SD431	kh.wong@xjtlu.edu.cn

课程占比

1. 5学分
2. 共2次assessments，占比共20%
3. 一个 2 小时的考试，占比 80%

Q&A

1. Name three typical type of data values in common data collections.

三种集合的典型数据类型值：整数（Integer）、字符串（String）、布尔值（Boolean）

2. Name three typical structures seen in common data collections.

三种典型的结构：数组（Array）、链表（Linked List）、栈（Stack）

3. Name three typical operations seen in common data collections.

三种典型操作：插入（Insert）、删除（Delete）、搜索（Search）。

4. Why do we learn data structure?

为什么我们学习数据结构？数据结构帮助我们有效地存储（storage）和管理（manage）数据，优化（optimize）数据处理的性能，是计算机科学的基础。

5. Why do we insist an algorithm must terminate?

为什么我们要求算法必须终止？

- 算法的主要目的是解决问题，而问题需要有一个明确的解决方案。
 - 如果算法不终止，它将无限运行，无法提供结果，从而无法达到解决问题的目的。
 - 终止性（Termination）是算法有效性（Effectiveness）和实用性（Practicality）的基本要求。

6. Why do we insist an algorithm must be precise?

为什么我们要求算法必须精确？精确的指令确保算法能够被正确执行，避免歧义（ambiguity）或错误。

7. Why instructions in an algorithm are written in a sequence?

为什么算法中的指令是按顺序编写的？顺序性（Sequentially）确保算法步骤的逻辑清晰，便于理解和执行。

■ [Week 1 Lecture 1]

数据结构是一种 **系统化的**(Systematic) 方式，用于组织数据集合以便**高效访问**(Efficiency access)。

每种数据结构都需要多种算法来处理其中的数据，因此，我们从以下方面研究数据结构

1. **属性** (properties)
2. **组织方式** (organisation)
3. **操作** (operations)

为什么我们需要数据结构？

虽然 原始类型（例如 `int`, `double` 这种）和 简单数组 可以满足基本需求，但数据结构提供了**更高级、更结构化的方式**来表达和处理复杂信息，使算法描述更清晰高效

我们将会探讨如下数据结构

1. 数组 **Arrays**
2. 列表 **Lists**
3. 栈 **Stacks**
4. 队列 **Queues**
5. 映射 **Maps**
6. 树 **Trees**
7. 图 **Graphs**

好的数据结构能优化效率，做出更好的软件。

因此，接下来我们探讨，有助于开发高质量软件的主要原则：**抽象**（abstraction）、**信息隐藏**（information hiding）、**封装**（encapsulation）。以及**静态结构**、动态结构（static, dynamic structures）之间的选择。

高质量软件3大原则

一个抽象的例子是，当我们查看导航时，地图绘制的是一片森林，而不是一颗一颗的树木。

[1] 抽象

我们可以将 抽象 视为一个过程（process）或一个实体（entity）。

- 作为过程，抽象指的是提取（extracting）一个项目或一组项目的基本细节，同时忽略非必要的细节。
- 作为实体，抽象指的是一个模型、视图或某种实际项目的表示。

在软件开发的背景下，我们又可以把 抽象 视为：

- 数据抽象（data abstraction）：确定数据存储和操作方式中哪些细节是重要的，哪些是不重要的。
- 过程抽象（procedural abstraction）：确定任务完成方式中哪些细节是重要的，哪些是不重要的。

抽象的关键：提取组件（components）的共性（commonality）并隐藏其细节。抽象通常关注对象/概念的**外部视图**（outside view）。

一个抽象的例子就是：**哈夫曼编码**（Huffman coding）。它是一种有效的数据编码方法，常用于通信。编码存储在**码本**（Code Book）中。在所有情况下，必须传输 **码本** 和 **编码数据**（Encoded data） 以进行解码。

哈夫曼编码

要了解霍夫曼编码是如何工作的，建议看 [这个视频（这个视频讲的非常好！）](#)。当然，接下来是用图文的形式简单介绍哈夫曼编码。

二进制编码

假设我们需要编码串 **A B A A C D C**，那么我们可以用二进制编码。在这里，字母出现的顺序分别是 **A B C D**， 因此他们可以用二进制编码：

字母	二进制
A	0
B	1
C	10
D	11

然后这个 **A B A A C D C** 就可以被加密为 **0 1 0 0 10 11 10** 也就是 **0100101110**。但是在解密的时候就会发生**歧义**。例如：组合 **BA** (**10**)可能被翻译为 **C**。这是因为 **B** 充当了 **C** 的前缀。

因此，为了解决歧义，引入了等长编码。

等长编码

字母	等长编码
A	00
B	01
C	10
D	11

这种编码的好处是一定不会出现前缀，每个字母对应的编码唯一，不会出现歧义。**00 01 00 00 10 11 10**

但是这种编码方式还是太长了。有什么编码方案能满足：

1. 编码出来最短
2. 没有歧义（能加密和唯一解密）

答案是：**哈夫曼编码**。

哈夫曼编码

哈夫曼编码的流程是：

1. 计算每个字母出现的次数。把这些次数当成**节点**
2. 每次相加两个最小的节点，构建成一个二叉树，其父节点是两个最小节点之和。

3. 继续步骤2，但是父节点也要算进去

这么看有点抽象，让我们来写一个例子。

还是这题，例如字符串 `A B A A C D C` 如何才能构建出哈夫曼编码？

步骤1：计算每个字母出现的次数

`A`：3次

`B`：1次

`C`：2次

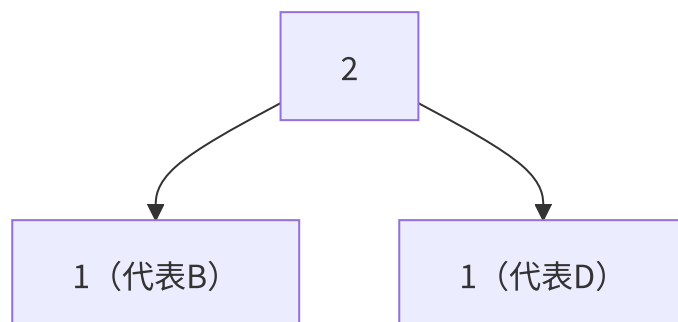
`D`：1次

步骤2：构建哈夫曼树

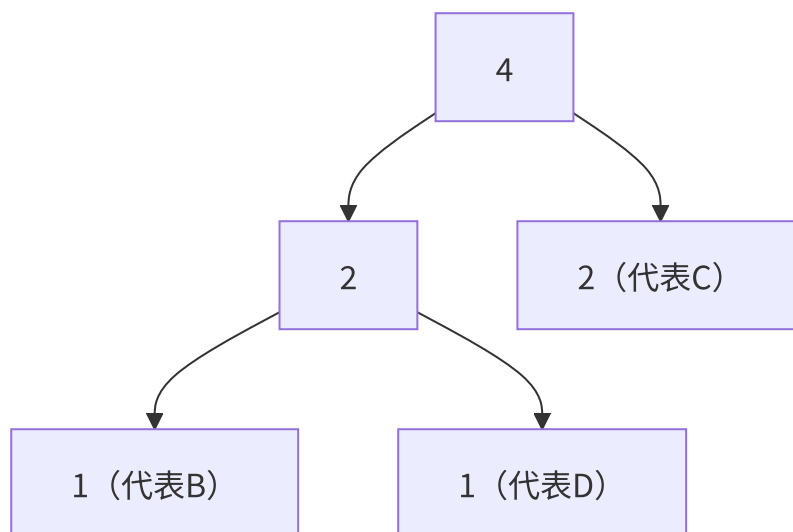
首先有四个节点，3，2，1，1，因此最小的两个节点是 1 和 1。



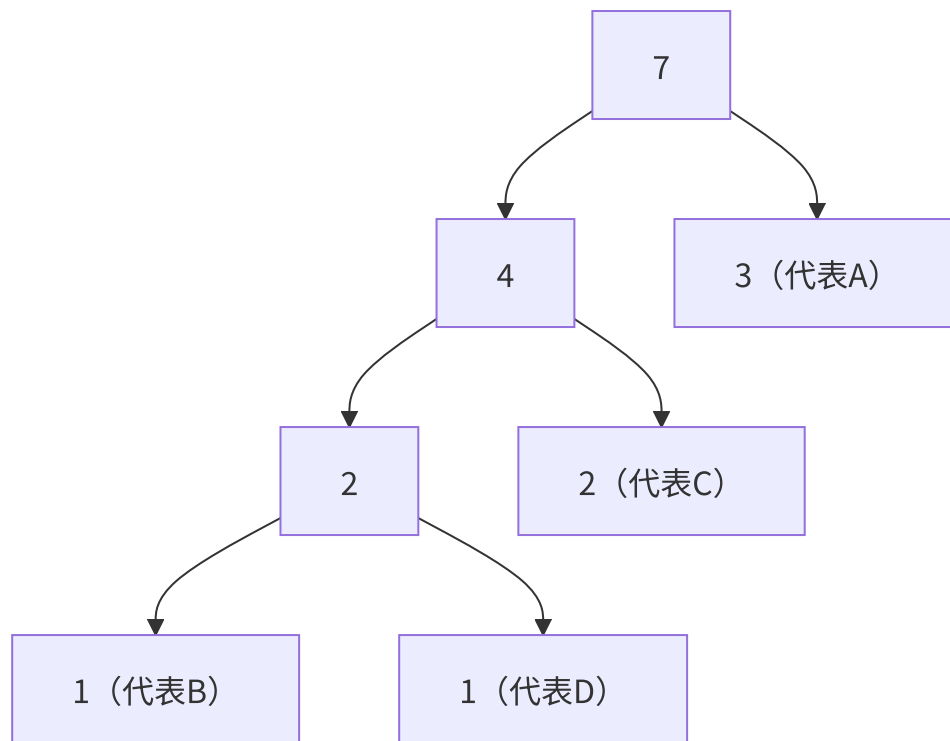
然后相加求出它们的父节点



接下来我们有还剩下三个节点：`3`（代表 `A`）、`2`（代表 `C`）、`2`（父节点）。因此最小的两个是 `2`（代表 `C`）和 `2`（父节点）。把他们拼接。



最后，我们剩下两个节点：`3`（代表 `A`）和 `4`（父节点）。因此，把他们拼接。



最后，我们定义左子树的边为0，右子树的边为1。这样，通过二叉树遍历的方式就能得出，A,B,C,D 的编码是：

A：1
B：000
C：01
D：001

因此，A B A A C D C 对应的哈夫曼编码是：1000110100101；

为什么哈夫曼编码能够解决前缀冲突（歧义）问题？因为所有的字母都在叶子节点上。假设A是B的前缀，那么A必须要在B的必经之路上。然而哈夫曼树的每一个字母都必然在叶子上，因此A不可能在B的必经之路上。这就解决了冲突问题。

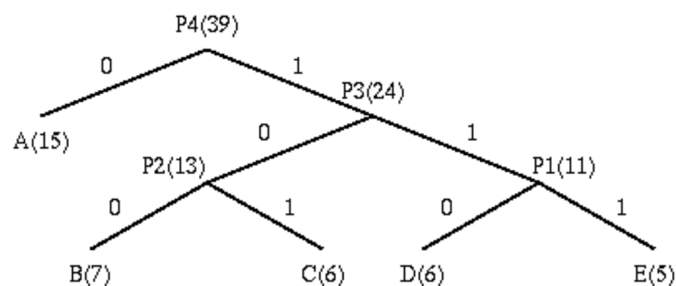
为什么哈夫曼编码能够最短？因为频率越大的字母越往树头靠，这样编码出来的就越短，压缩率更大。

值得注意的是，最开始我们操作B和D的时候，左右是没有讲究的。虽然左右会影响哈夫曼编码出来的值，但是并不影响其性质。也就是说，**对于一个串，哈夫曼编码的值不是唯一的。**

Subtotal (# of bits) 指的是**每个符号在编码后所占用的总位数**。实际上计算公式如下：

$$subtotal = frequency * length$$

这里的 frequency 就是频率，length 就是路径长度（就是走了多少步）。例如：



Symbol	Count	Code	Subtotal (# of bits)
A	15	0	15
B	7	100	21
C	6	101	18
D	6	110	18
E	5	111	15
TOTAL (# of bits):			87

这里的 A 对应的解码是 0，那么其 `subtotal = 15 * 1 = 15`。

B 的 `subtotal = 7 * 3 = 21`；如此往复。

哈夫曼编码使用了**优先队列**(Priority queues)和**二叉树**(Binary trees)。

[2] 信息隐藏

信息隐藏：指模块的使用者只需了解该模块的必要细节（通过**抽象**识别）

users of a module need to know only the essential details of this module

而抽象帮助我们识别出哪些细节重要，哪些不重要。

信息隐藏旨在将内部数据或信息屏蔽，避免被直接操控（direct manipulation）。

信息隐藏示例：汽车与驾驶员

汽车通过为驾驶员提供一个**标准化接口**（如踏板、方向盘、变速杆、信号灯、开关等），展示了信息隐藏的概念。这个接口隐藏了汽车内部的复杂机制（如发动机、变速箱、电子系统等）。驾驶员通过培训和考试学习使用这个接口，使他们能够操作任何汽车，而无需了解其内部工作原理。

或者java中，通过**封装**的方法来进行信息隐藏。

[3] 封装

与抽象类似，我们可以将**封装**视为一种**过程**或一种**实体**

- 作为一种过程，封装意味着：将一项或多项（数据/函数）纳入一个**容器**中的行为。
- 作为一种实体，封装意味着：一个**容器**，其中包含一项或多项（数据/函数）。（该容器内部与外部的分隔有时被称为“**墙**(Wall)”或“**屏障**(Barrier)”）

面向对象（O-O, Object-Orient）中的封装

在面向对象编程中，封装是指将对象正常运行所需的全部资源（即方法和数据）包含在对象**内部**。其核心思想是：不要告诉我你是怎么做的；只要让我知道你能做什么！

通信 (Communication) 中的封装

在通信领域，封装是指将一个数据结构包含在另一个结构内，从而使第一个数据结构被隐藏。

例如，一个 TCP/IP 格式的数据包可以被封装在一个 ATM 帧中。封装的核心目的是将复杂的细节隐藏在一个更简单或标准化的形式中，以便于传输和处理。

[4] 总结

抽象 (Abstraction)、**信息隐藏 (Information Hiding)** 和 **封装 (Encapsulation)** 是不同的概念，但它们彼此相关。

- **抽象** 是一种帮助我们识别某个模块中哪些信息对用户是重要的、哪些是不重要的技术。
- **信息隐藏** 是一项原则 (Principle)，即所有不重要的信息都应该对用户隐藏。
- **封装** 则是将信息打包的技术，目的是隐藏那些应该隐藏的内容，同时暴露那些需要可见的内容。

抽象和**封装**在**信息隐藏**的指导原则下，我们享有以下优势：

1. 更简单、模块化的程序，更易于设计和理解
2. 消除或最小化**直接操作**(direct manipulation)数据带来的副作用
3. 让错误局部化（只有类上定义的方法才能操作类的数据），从而支持局部化测试
4. 程序模块更易于阅读、修改和维护 (maintain)

[5] 静态和动态数据结构的选择

除了**时间**和**空间**效率之外，选择数据结构的另一个重要标准是:能存储的数据项数量是否可以根据我们的需求进行调整，或者是固定的。

动态数据结构：可以在运行时根据当前需求进行增长或缩小。

例如用于模拟交通流量的结构。

静态数据结构：在创建时就是固定的

例如，用于存储邮政编码或信用卡号码的结构（这些数据具有固定格式）

注意，静态（或动态）的是**结构**，而不是数据。

因此，在静态数据结构中，存储的数据可以随时间变化，但结构是固定的。

例如C++中的数组。一般而言，数组长度不能改变。

示例

静态数据结构的一个示例是**数组**：

```
1 | int[] a = new int[50];
```

这行代码分配了可以容纳50个整数的内存空间。

Java提供了 `new` 一词来向编译器/解释器指示需要分配多少空间以及哪种数据类型空间。

即使我们编写以下代码，数组也总是动态分配的（Dynamically allocated）：

```
1 | int[] a = {10, 20, 30, 40};
```

然而，数组的大小是固定的。但问题是，什么叫**动态分配**？

在Java中，数组即使在编译时定义大小（例如 `int[] a = {10, 20, 30, 40};`），它们仍然是在**运行时分配内存**的。这意味着数组的内存空间是在程序**运行期间**通过堆内存动态获取的（`new`出来的都在堆内存中），而不是在编译时静态地确定和分配。这与静态内存分配的语言（如C中的静态数组）不同，后者在编译时已经确定了内存分配。

问题：Java数组是静态还是动态数据结构？

Java数组从**内存分配（Memory allocated）**的角度来看是**动态的**，因为它们的内存是在运行时分配的。

然而，数组的大小在初始化时（at the time of initialize）是固定的，因此在大小上表现出静态特性。

[6] 静态数据结构优点和缺点

优点

- 易于指定（Ease of specification）。编程语言通常提供一种简便的方式来创建**任意大小**的静态数据结构。
没有内存分配开销。

没有内存分配开销指：静态数据结构在运行时**不需要动态分配或释放内存**，从而避免了与动态内存管理相关的性能开销。

缺点：

由于静态数据结构的大小是固定的：

1. 不存在可以用来扩展静态结构的操作；
2. **如果要对静态数据结构进行扩展（例如增加元素），就需要为其分配额外的内存空间**，而分配内存的操作是需要时间的，因此会产生时间和性能上的消耗
3. 一方面，**必须确保有足够的容量**；另一方面，如果存储的数据项数量较少，那么静态数据结构的一部分将保持为空，但这些内存已经被分配，无法用于存储其他数据

[7] 动态数据结构的优点和缺点

优点

1. **无需事先知道确切的数据项数量**
2. **内存空间的高效利用**
 1. 在需要添加数据项时（如果这些数据项无法存储在当前结构中），可以扩展动态数据结构的大小。
 2. 在动态数据结构中存在未使用的空间时，可以收缩其大小。这样，结构总是能够保持恰到好处的大小，不会浪费任何内存空间。

缺点

1. **内存分配/释放开销**：这意味着在使用动态数据结构（如链表、堆栈或队列）时，系统需要分配新的内存空间来存储增加的数据，或者释放不再需要的内存空间。这些操作会产生一定的性能开销。
2. **每当动态数据结构增长或缩小时，分配给数据结构的内存空间必须增加或减少（这需要时间）**：这意味着随着动态数据结构的扩展或缩小，系统需要调整内存分配，这过程涉及到重新分配内存并可能移动数据，因此会花费额外的时间。

Q&A

1. **空间效率(Space efficiency)和时间效率(Time efficiency)**都是用来评估算法（以及数据结构）性能的指标。（对还是错？）

这句话是正确的。空间效率衡量的是算法运行所需的最大内存空间，而时间效率衡量的是算法完成任务所需的时间。这两者都是评估算法性能的重要标准。

2. 动态数据结构通常更具空间效率。（对还是错？）

这句话是错误的。动态数据结构在内存使用上并不总是比静态数据结构更高效。相反，因为动态数据结构需要**额外的内存来存储指针或链接**，而且可能会导致**内存碎片**，所以静态数据结构在某些情况下可能更节省内存。

3. 静态数据结构通常更具有时间效率。（对还是错？）

这句话是正确的。静态数据结构（如数组）由于内存分配是预先确定的，不需要频繁的内存分配和释放操作，因此在访问和操作数据时通常更快，因此具有更高的时间效率。

4. 信息隐藏是指：软件组件的用户只需要知道**如何初始化**和访问组件的必要细节，而不需要了解实现的细节。（对还是错？）

对的。比如你开车，只需要知道如何初始化（启动发动机）和如何把握方向盘这些即可。不需要知道内燃机是如何工作的，这属于细节方面的。

总结

数据结构：

数据结构是计算机中**组织和存储数据的方式**，决定了数据的访问和处理效率。

抽象：

抽象是指隐藏复杂的实现细节，只关注问题的核心概念，是设计和理解数据结构的重要方法。

抽象的应用：

1. 数据挖掘（Data Mining）
2. 大数据分析（Big Data Analytics）
3. 数据库知识发现（Knowledge Discovery in Databases / KDD）

哈夫曼编码与动态队列：

哈夫曼编码是一种用于**数据压缩的技术**，动态队列（Dynamic queues）是一种根据数据量动态调整大小的队列结构。

信息隐藏：

信息隐藏是指将实现**细节**与用户接口**分离**，用户只需知道如何使用组件，而不必了解其实现。

封装：

封装是将数据和操作数据的方法绑定在一起，**隐藏内部细节**，确保数据的安全性和一致性。

空间与时间效率：

空间效率是指算法所需的内存资源，时间效率是指算法执行所需的时间，两者都是评估算法性能的关键指标。

静态与动态数据结构：

静态数据结构（如数组）的大小在编译时确定，而动态数据结构（如链表）的大小可以根据需要动态调整，但动态数据结构通常会牺牲一些空间或时间效率。

■ [Week 2 Lecture 2]

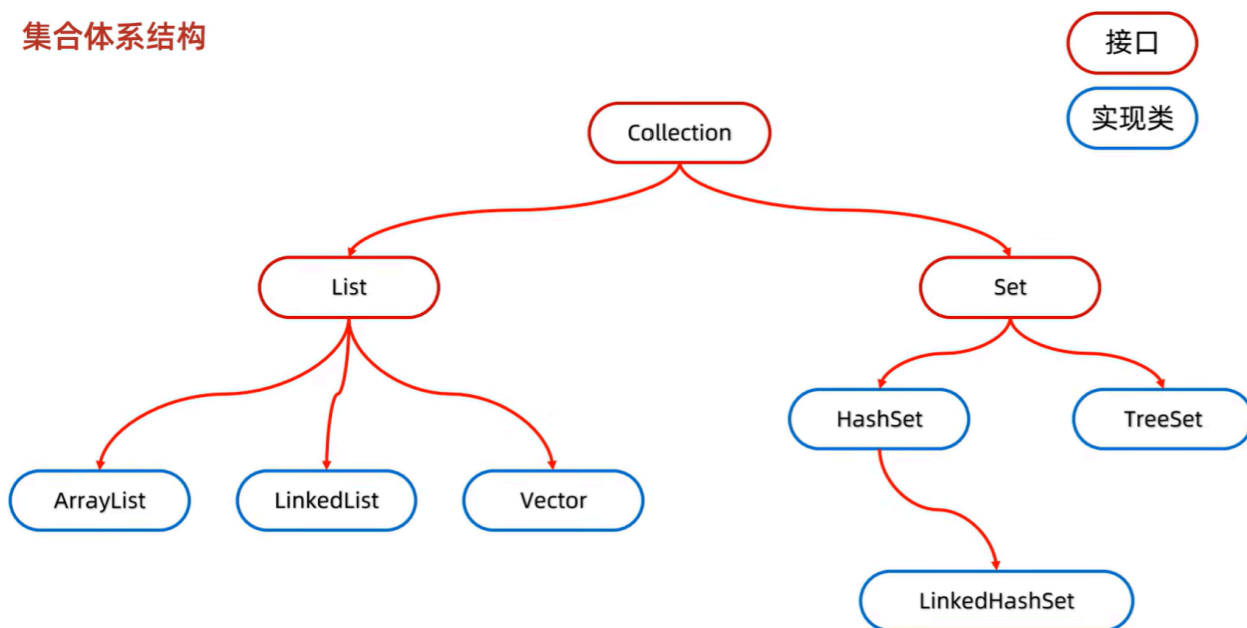
标题：使用 Java `Collection` 库

Lecture 2主要介绍了 Java 的一些功能。PPT非常乱，所以Lecture 2 进行了一些调整，至少看起来有些逻辑

1. 两种集合类型
2. 学会使用 Java 库进行编程
3. 集合
4. 使用对象列表（Lists of Objects）进行编程

[0] 非常重要的一张图

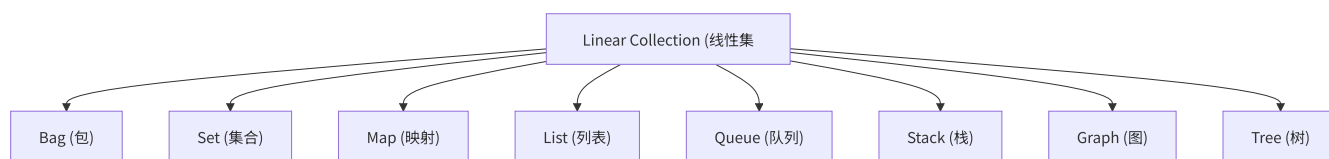
集合体系结构



这张图PPT上没有。而且PPT讲的东一块西一块，如果你能时刻看懂这张图，可能会对你的学习有所帮助。

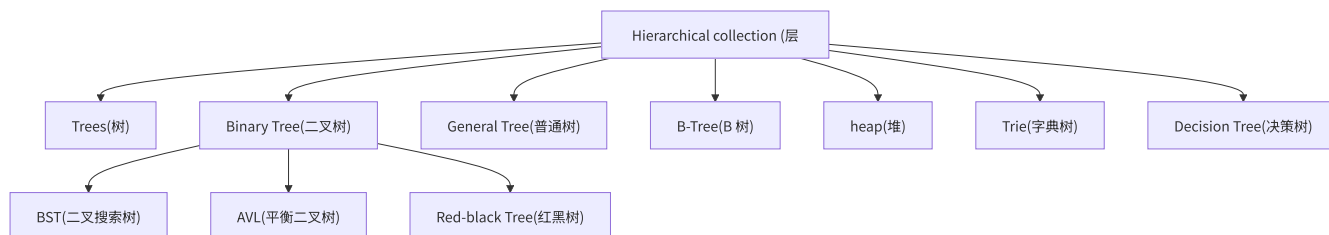
[1] 两种集合类型

1. 线性集合(Linear Collection)的种类有很多：



使用线性集合来编程，实现数据的搜索与排序，实现链表数据结构与“指针”

2. 层次化集合（Hierarchical collections）的种类也有很多：



PPT中只列出Tree、Binary Tree、General Tree。但其余的数据结构也很重要。例如决策树用于机器学习。能利用AVL树加泛型 `<E>` 在Java中创建集合。

使用层次化结构，实现树的构建、搜索、遍历（Traversing）...

[2] 学会使用库进行编程

因为例如GUI、网络程序，规模过大无法从头搭建，因此学会使用库进行编程尤为重要

Java 提供了很多标准工具类。

`java.util` 集合类

`java.io` 输入输出类

`javax.swing` `java.awt` 用于 GUI 程序的大型类库...

泛型

泛型（Generics）

接下来用一个例子来介绍泛型 `<E>`：

```
1 public class MyLists<E>{           // 定义一个泛型类
2     private ArrayList<E> list = new ArrayList<>();
3
4     public E getObject(int index){ // 返回第 index 个元素
5         return list.get(index);
6     }
7     public void add(E element) // 添加元素
8     {
9         list.add(element);
10    }
11 }
```

`MyLists` 中什么都能存，所以 `getObject` 方法返回的类型不确定，所以就填 `<E>`。但是一旦 `MyLists` 的对象被创建，虚拟机就知道这个泛型到底代表着什么。例如 `Mylists<Integer> list = new Mylists();` 虚拟机就知道这个泛型 `E` 代表 `Integer`。如果在编译前试图插入非 `Integer` 数据，IDE就下标会红线报错。

当然如果不写，那就是默认为 `Object`。这时候集合里面什么都能放。不会有任何限制。

因此这就是泛型的作用。接下来是对应的例子：

```
1 public static void main(String[] args) {
2     // 没有指定数据类型，默认Object什么都能放
```

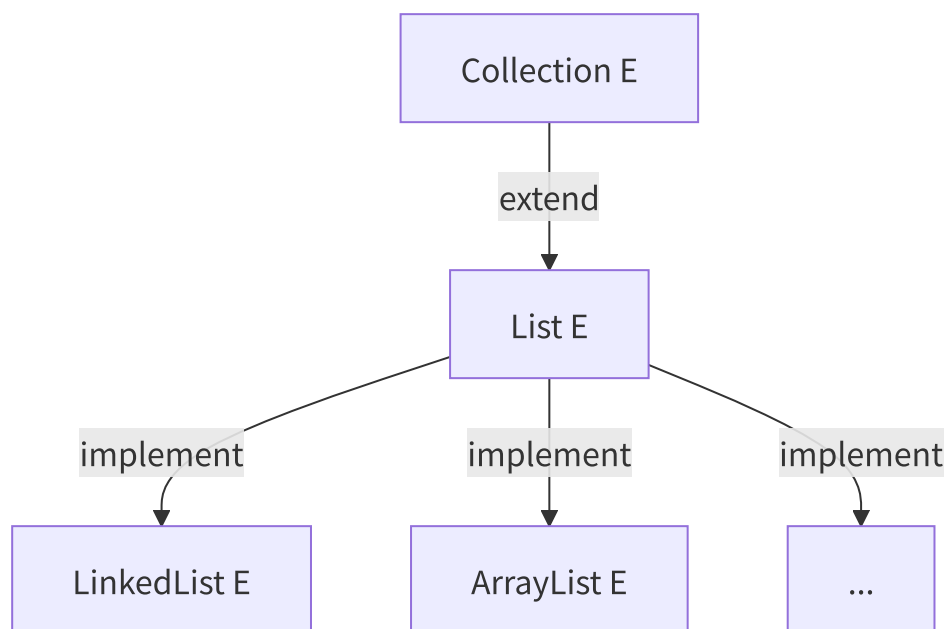
```

3      MyLists listNoSpecific = new MyLists();
4
5      listNoSpecific.add(1); // 添加 Integer
6      listNoSpecific.add(new Student("zhangsan",13)); // 添加自定义引用数据类型
7      System.out.println(listNoSpecific.getObject(0));
8      System.out.println(listNoSpecific.getObject(1));
9      // 输出:
10     // 1
11     // Student{name='zhangsan', age=13}
12
13     // 如果确定了数据类型。那么泛型E就会记住，你这个对象只能存Integer类型
14     MyLists<Integer> listSpecific = new MyLists<>();
15
16     listSpecific.add(1);           // 可插入，不报错
17     listSpecific.add(new Student("zhangsan",13)); // 报错!!
18 }

```

接口和实现类

Java 中存在接口继承接口。例如 `Collection <E>` 下就被 `List<E>` 继承。而 `List<E>` 接口能被实现为 `ArrayList<E>` 和 `LinkedList<E>`。



Java 中的 `Collection`（接口）等类非常抽象（Abstract），因为他实现类接受泛型 `<E>`（也就是什么数据类型都能存）。在这里，`<E>` 相当于一个记录变量类型的变量。

```
1 public interface Set<E> extends Collection<E> {
2     public void add(??? item); /*...描述...*/
3     public void remove(??? item); /*...描述...*/
4     public boolean contains(??? item); /*...描述...*/
5     ...
6     // （Java Set 接口中还有许多其他方法）
7 }
```

Java 接口中**没有构造函数（无法实例化），没有字段（无法储存数据），没有方法体（空的，等实现类 @override）。**

作为接口，其对应的实现类对象有很多。

- 1. 不关心元素的类型是什么。
- 2. 不关心它们内部如何存储或操作。

接口	说明
Collection	包（最通用的集合类型）
List	有序集合
Set	无序集合，不允许重复元素
Queue	有序集合，访问受限（从一端添加，从另一端移除）
Map	键值对 key-value （或映射 Mapping）

实现类	类名
List 类	ArrayList, LinkedList, Vector, ...
Set 类	HashSet, TreeSet, ...
Map 类	HashMap, TreeMap, ...
其他	...

[3]Q&A

1. Name 3 type of collections that can be implemented under Java Programming with Linear collections

在 Java 编程中，可以被实现（这里的实现是实现接口 implement interface的意思）的线性集合类型的三种类型是：

答案：列表（List）、队列（Queue）、栈（Stack）、...（参考 **[1] 两种集合类型** 中的 线性集合）

2. Name 3 operations that can be implemented under Java Programming with Linear collections

Java 中能被实现的线性集合的三种操作是：

答案：add() 添加元素，remove() 删除元素，get() 获取元素。

3. Name 2 type of collections that can be implemented under Java Programming with Hierarchical collections

在 Java 编程中，可以被实现的层次化集合类型的两种类型是：

答案：

1. Tree（基于树的集合）
2. TreeMap（基于树的映射）

4. Name 4 operations that can be implemented under Java Programming with Hierarchical collections

Java 中能被实现的层次化集合的四种操作是

答案：put() 添加键值对，remove() 删除键值对，get() 获取值，traverse() 遍历集合。

5. What is a software “library”?

什么是软件“库”？

答案：软件库是一组预先编写好的代码、函数或类，供开发者在编写程序时调用，以减少重复工作和提高开发效率。例如Java的各种包，比如 java.lang 是 Java 的标准库之一。

6. Define Java “Package”.

定义 Java 的包

答案：Java 包（Package）是用于组织类（Class）和接口（Interface）的命名空间，类似于文件夹。它用于避免命名冲突，并提供访问控制。

例如，

```
1 class Tools{
2     protected static String HelloWorld(){
3         System.out.println("HelloWorld")
4     }
5 }
```

Tools 中的 HelloWorld() 方法能被所有的继承子类（extend 的子类）和所有在一个包下的类访问。而其他包下的非子类则不能访问。

7. Name Java's IO library.

Java 的 IO 库是什么？

答案：Java 的 IO 库是 java.io，用于处理输入输出操作，如文件读写、流操作等。

8. Name Java's GUI library.

Java 的 GUI 库是什么？

答案：Java 的 GUI 库是 java.awt（抽象窗口工具包）和 javax.swing（Swing 框架）。

9. What is the Java statement to include package or class into your program?

用于将包或类导入到程序中的 Java 语句是什么？

答案：使用 import 语句，例如：

```
1 import java.util.ArrayList; // 导入 ArrayList 类
2 import java.util.*;         // 导入 java.util 包中的所有类
```

■ [Week 2 Lecture 3]

Lecture 3 讲了 Collection 和 List，以及迭代器。

上个 Lecture 中介绍了 `Collection<E>` 接口，以及继承它的接口 `List<E>`。子接口必须拥有父接口的所有方法。同时可以适当增加属于自己的方法。

对于 `List` 这些线性集合，实际上是有索引的。但是对于一些 `set` 集合，他们必须使用其他方法进行遍历（比如迭代器）。PPT上只说了用 `Iterator`（迭代器）遍历，实际上，对于 `collection` 下的实现类，一共有三种遍历方式。

三种遍历方式

1. 迭代器遍历
2. 增强 `for` 遍历
3. Lambda表达式遍历

迭代器遍历

迭代器不依赖索引，其类叫 `Iterator`。

迭代器遍历时，不能用集合的方法进行增加或者删除，会发生并发修改异常

`ConcurrentModificationException`。（非常重要，但是PPT没提）

如果需要删除，那么就用迭代器提供的 `.remove()` 方法进行删除。如果要添加元素，现在还没有办法。

Collection集合获取迭代器

方法名称	说明
<code>Iterator<E> iterator()</code>	返回迭代器对象，默认指向当前集合的0索引

Iterator中的常用方法

方法名称	说明
<code>boolean hasNext()</code>	判断当前位置是否有元素，有元素返回true，没有元素返回false
<code>E next()</code>	获取当前位置的元素，并将迭代器对象移向下一个位置。

- ★ `.hasNext()` 这个是判断当前位置是否有元素，而不是下一个是否有元素。
- ★ `.next()` 这个会做2个事情，一个是获取当前元素，然后移动指针到下一位。

```

1 Collection<String> coll = new ArrayList<>();
2 coll.add("123");
3 coll.add("456");
4 coll.add("789");
5 coll.add("000");
6 Iterator<String> it = coll.iterator(); // 新建迭代器
7 while (it.hasNext()){
8     System.out.println(it.next());
9 }

```

增强 for 遍历 (推荐)

- 增强 for 的底层就是迭代器，为了简化迭代器的代码书写的。
- 它是JDK5之后出现的，其内部原理就是一个 `Iterator` 迭代器
- 所有的单列集合和数组才能用增强for进行遍历。

格式：

```

1 for (元素的数据类型 变量名 : 数组或者集合) {
2 }

```

示例：

```

1 for (String s : list) {
2     System.out.println(s);
3 }
4 // list.for 按下tab自动补全

```

细节：修改增强 for 的第三方变量不会改变原本集合中的数据。

Lambda表达式遍历

得益于JDK 8开始的新技术Lambda表达式，提供了一种更简单、更直接的遍历集合的方式。

方法名称	说明
<code>default void forEach(Consumer<? super T> action)</code>	结合lambda遍历集合

这里的Consumer是接口，我们可以利用匿名内部类的形式。

```

1 coll.forEach(new Consumer<String>() // 接口
2     // 匿名内部类
3     {
4         @Override
5         public void accept(String s) { // 这里的s就是依次表示集合中的每个数据。
6             // 这里直接操作s即可
7         }
8     }
9 });

```

`forEach` 的底层原理，其实也会自己遍历集合然后得到每一个元素，然后交给`accept`方法。

简化后的匿名内部类，也就是Lambda表达式可以这样写：

```
1 | coll.forEach(s -> System.out.println(s));
```

■ [Week 3 Lecture 4]

[1] 包 Bags

包 (Bags)

包是一种集合，具有以下特性：

1. 不维护结构或顺序
2. 无访问限制（可随时访问任何元素）
3. 允许重复元素

基本操作：

1. `add(value)` → 如果集合被修改则返回 `true`
2. `remove(value)` → 如果集合被修改则返回 `true`
3. `contains(value)` → 如果值在包中则返回 `true`
（使用 `equals` 进行测试）
4. `findElement(value)` → 如果值在包中则返回匹配的元素

附加操作：

```
size(), isEmpty(), iterator(), clear(),  
addAll(collection), removeAll(collection),  
containsAll(collection), ...
```

包的应用

何时使用包？ 当集合不需要排序，且可能包含重复元素时：

1. 当前登录用户的集合（可能有重复吗？）
2. 书籍收藏中的书籍（可能有重复吗？）
3. ...

包没有标准的实现！

[2] 集合 Set

集合 (Set ADT)

集合是一种具有以下特性的集合：

1. 不维护结构或顺序
2. 无访问限制（可随时访问任何元素）
3. 唯一特性是排除重复元素

操作： （与包类似，但行为不同）

1. `add(value)` → 如果值被添加则返回 `true`（即没有重复）
2. `remove(value)` → 如果值被移除则返回 `true`（值在集合中）
3. `contains(value)` → 如果值在集合中则返回 `true`
4. `findElement(value)` → 如果值在集合中则返回匹配的元素

`findElement(value)` → 如果值在集合中则返回匹配的元素

这里的“匹配的元素”指的是集合中与给定值 `value` 相等的元素。由于集合不允许重复元素，如果 `value` 存在于集合中，`findElement(value)` 会返回集合中的那个元素（即与 `value` 相等的元素）。如果 `value` 不在集合中，则返回 `null` 或某种表示未找到的值。

例如：

- 假设集合 `S = {1, 2, 3, 4, 5}`，调用 `findElement(3)` 会返回 `3`，因为 `3` 在集合中。
- 调用 `findElement(6)` 会返回 `null`，因为 `6` 不在集合中。

这个操作通常用于检索集合中具体的元素，而不是仅仅检查元素是否存在（`contains(value)` 用于检查存在性）。

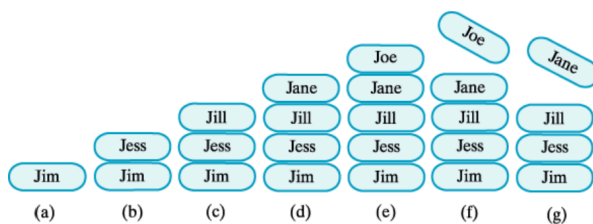
5. ...

集合与列表一样常见

[3] 栈 (Stack)

- 栈根据元素添加的顺序来组织条目
- 添加操作总是在一端进行，称为栈顶(Top)
- 最近添加的元素始终位于栈顶

(a) 执行 `push` 操作，添加 Jim;
(b) 执行 `push` 操作，添加 Jess;
(c) 执行 `push` 操作，添加 Jill;
(d) 执行 `push` 操作，添加 Jane;
(e) 执行 `push` 操作，添加 Joe;
(f) 执行 `pop` 操作，检索并移除 Joe;
(g) 执行 `pop` 操作，检索并移除 Jane;



栈 (Stacks) 是一种特殊的列表 (List)：

- 它是一个值的序列（“序列”意味着元素是按顺序排列的）。
- 受限的访问方式：只能从一端进行添加、获取和移除操作。
- 存在栈接口及其不同的实现（如 `ArrayStack`、`LinkedStack` 等）。

在 Java 集合库中：

- `Stack` 是实现了 `List` 接口的类。
- 它提供了额外的操作：`push(value)`、`pop()` 和 `peek()`：
 - `push(value)`：将值压入栈顶。
 - `pop()`：移除并返回栈顶的元素。

- `peek()`：返回栈顶的元素，但不移除它。
- 此外，它还支持其他列表操作。

Stack 类

java.util 包中 Stack 类的方法

```
1 // 将元素 item 压入栈顶
2 public void push(Object item);
3
4 // 弹出并返回栈顶元素
5 public Object pop();
6
7 // 返回栈顶元素但不弹出
8 public Object peek();
9
10 // 判断栈是否为空
11 public boolean isEmpty();
12
13 // 清空栈
14 public void clear();
```

关于 StackInterface

```
1 public interface StackInterface
2 {
3     /** 任务：将新条目添加到栈的顶部。
4      * @param newEntry 要添加到栈中的对象 */
5     public void push(Object newEntry);
6
7     /** 任务：移除并返回栈顶部的对象。
8      * @return 栈顶部的对象，如果栈为空则返回 null */
9     public Object pop();
10
11     /** 任务：获取栈顶部的对象。
12      * @return 栈顶部的对象，如果栈为空则返回 null */
13     public Object peek();
14
15     /** 任务：判断栈是否为空。
16      * @return 如果栈为空则返回 true */
17     public boolean isEmpty();
18
19     /** 任务：移除栈中的所有条目 */
20     public void clear();
21 } // 结束 StackInterface
22 Specifications of the ADT Stack
23 • Specification of a stack of objects
```

[4] 栈的应用

- 处理结构化（嵌套）数据文件：
 - 例如，读取带有结构化标记的文件（如 HTML、XML 等）。
- 程序执行：
 - 例如，处理子任务后返回到先前的任务。
- 编辑器中的撤销操作：
 - 栈用于记录操作历史以支持撤销功能。
- 表达式求值：
 - 例如，计算表达式 $(6 + 4) * \left((12.1 \times \sin(15)) - \left(\frac{\cos(20)}{38} \right) \right)$ 。

Example: 如何确保网页文档中的 XML/HTML 标签正确嵌套？

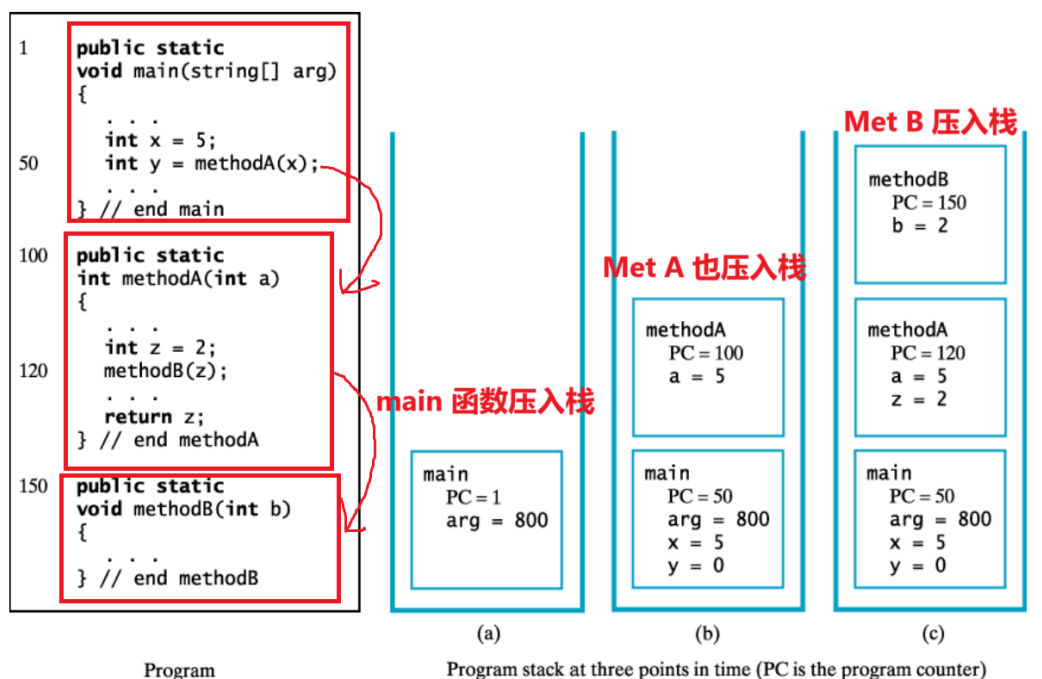
答案：使用栈！

程序执行中的程序栈

当一个方法被调用时

- 运行时环境 会创建一个**活动记录(activation record)**来展示方法在执行期间的状态
- 活动记录被压入程序栈（Java 栈）
- 栈顶属于**当前正在执行的方法**，栈中的下一个记录属于调用当前方法的方法

这里的**方法**指的是Java中的函数，或者叫做method，就例如 `public static void calculate(int a, int b){return a+b}` 这种



如果无人调用Met B那么就弹出栈，再弹出A，最后弹出main

递归方法

一个递归方法进行多次递归调用时：

- 会在程序栈中放置多个活动记录
- 这解释了为什么递归方法可能会占用大量内存
- 可以通过使用栈将**递归recursion**替换为**迭代iteration**

用于表达式求值

例如表达式： $(6 + 4) * ((12.1 * \sin(15)) - (\frac{\cos(20)}{38}))$

它是如何工作的？

使用栈处理代数表达式

- 检查中缀代数表达式中的**括号（圆括号、中括号、大括号）是否平衡**
- 将中缀表达式转换为后缀表达式
- 求值后缀表达式
- 求值中缀表达式

括号匹配

```
1 public static boolean isBalanced(String string) {
2     Stack<Character> stack = new Stack<Character>();
3     for (int i = 0; i < string.length(); i++) {
4         switch (string.charAt(i)) {
5             case '{' -> {
6                 stack.push('{'); // 左括号统一入栈
7             }
8             case '}' -> {
9                 if(stack.peek()=='{') // 右括号则检查栈顶是否是对应左括号，是的话弹出
10                    stack.pop();
11            }
12            case '[' -> {
13                stack.push '[';
14            }
15            case ']' -> {
16                if(stack.peek()=='[')
17                    stack.pop();
18            }
19            case '(' -> {
20                stack.push('(');
21            }
22            case ')' -> {
23                if(stack.peek()=='(')
24                    stack.pop();
25            }
26        }
27    }
28 }
```

```
26         }
27     }
28     return stack.isEmpty();
29 }
```

后缀表达式

*你需要直到怎么用草稿纸把中序表达变成后续表达即可。

中缀表达式和后缀表达式是数学表达式的两种不同表示方式。

中缀表达式 (Infix Expression):

中缀表达式是我们通常书写数学表达式的方式，其中运算符位于两个操作数之间。例如：

$(a + b) * c$

后缀表达式 (Postfix Expression):

后缀表达式（也称为逆波兰表达式）中，运算符位于两个操作数之后。例如，上述中缀表达式的后缀表达式形式为：

$ab + c*$

如何将中缀表达式转换为后缀表达式:

可以通过使用栈的数据结构来实现转换。以下是转换的步骤：

1. 初始化一个空栈用于存放运算符，初始化一个空列表用于输出后缀表达式。
2. 从左到右扫描中缀表达式。
3. 如果遇到操作数，直接将其添加到输出列表。
4. 如果遇到左括号"(", 将其压入栈中。
5. 如果遇到右括号")", 则从栈中弹出元素并添加到输出列表，直到遇到左括号为止。左括号不添加到输出列表，而是从栈中弹出。
6. 如果遇到运算符，则：
 - 假设当前遇到的字符 `currentChar` 和 `stack` 顶部的字符 `peek`。重复以下步骤：
 - 如果 `currentChar` 和 `peek` 是同级的（加减同级，乘除同级），弹出 `peek`，输出列表添加 `peek`
 - 如果 `currentChar` 级别小于 `peek`，弹出 `peek`，输出列表添加 `peek`
 - 直到遇到`peek`级别比自己小，或者遇到 `(`，或者 `stack.isEmpty()==true`
7. 扫描完中缀表达式后，将栈中剩余的运算符依次弹出并添加到输出列表。
8. 输出列表即为后缀表达式。

示例:

将中缀表达式 $(a + b) * c$ 转换为后缀表达式:

1. 初始状态：栈 = [], 输出列表 = []
2. 遇到 '(', 压入栈中：栈 = ['('], 输出列表 = []
3. 遇到 'a', 添加到输出列表：栈 = ['('], 输出列表 = ['a']
4. 遇到 '+', 压入栈中：栈 = ['(', '+'], 输出列表 = ['a']
5. 遇到 'b', 添加到输出列表：栈 = ['(', '+'], 输出列表 = ['a', 'b']

6. 遇到 ')', 弹出栈中元素 '+' 并添加到输出列表, 弹出 '(': 栈 = [], 输出列表 = ['a', 'b', '+']
7. 遇到 '*', 压入栈中: 栈 = ['*'], 输出列表 = ['a', 'b', '+']
8. 遇到 'c', 添加到输出列表: 栈 = ['*'], 输出列表 = ['a', 'b', '+', 'c']
9. 扫描完表达式, 弹出栈中剩余的 '*' 并添加到输出列表: 输出列表 = ['a', 'b', '+', 'c', '*']

最终的后缀表达式为: $ab + c*$

后缀表达式更容易处理: 后缀表达式的计算顺序是固定的, 不需要考虑运算符的优先级和括号的影响

练习:

写出这几个中缀表达式的对应后缀表达式

```
1 System.out.println(compile("{3+[d-7*(g+5)]/w} "));
2 System.out.println(compile(" (a+n)*(b-8*m)"));
3 System.out.println(compile("b/v^7"));
4 System.out.println(compile("[ (4+b)-2 ]"));
```

答案:

```
1 3d7g5+*-w/+
2 an+b8m*--*
3 bv7^/
4 4b+2-
```

代码实现查看 INT102 文件夹下 [Algorithm.pdf](#)

[5] 图 Map

数据的集合, 但不是单个值的集合:

- 映射 (Map) = 键到值的对的集合
- 受限的访问: 通过键获取值
- 不允许重复的键
- 有多种实现, 最常见的是哈希映射 (HashMap)

在声明和构造时, 必须指定两种类型:

- 键的类型和值的类型

```
1 private Map<String, Integer> phoneBook; // K, V
2 phoneBook = new HashMap<String, Integer>(); // K, V
```

• 核心操作:

- `get(key)`, → 返回与键关联的值 (如果不存在则返回 `null`)
- `put(key, value)`, → 设置与键关联的值 (并返回旧值, 如果有的话)
- `remove(key)`, → 移除键及其关联的值 (并返回旧值, 如果有的话)

- `containsKey(key)`, → 返回布尔值
- `size()` 返回大小

应用

字数统计：给出一个文本，查找哪个文字的出现频率最高。这里我们把“文本”定义为一个 `char`。我们尝试用 `Map` 来实现

```
1 public class Frequency {
2     public static void main(String[] args) {
3         String s = "Once upon a time, in a small village, a curious cat named Max often
4         wandered the streets. One day, he found a shiny key near an old oak tree. Intrigued,
5         Max carried the key home. His owner, Mia, recognized it as the key to her grandmother's
6         antique chest. Inside, they discovered a treasure map. Together, Max and Mia embarked
7         on an adventure to find the hidden treasure, learning the true value of curiosity and
8         teamwork along the way.";
9         compile(s);
10        getFrequency();
11    }
12    private static Map<Character,Integer> map = new HashMap<>();
13    public static void compile(String string){
14        StringBuilder sb = new StringBuilder(string);
15        for (int i = 0; i < sb.length(); i++) {
16            char currentChar = sb.charAt(i);
17            if(!map.containsKey(currentChar)) // 如果这个Map不存在这个char，就存入
18                map.put(currentChar,1);
19            else
20                map.put(currentChar,map.get(currentChar)+1); // 如果存在，就把Value ++;
21        }
22    }
23    public static void getFrequency(){
24        StringBuilder sb = new StringBuilder();
25        int maxValue = 0;
26        char key='0';
27        for( Map.Entry<Character,Integer> entry : map.entrySet()){
28            if(entry.getValue() > maxValue)
29            {
30                maxValue = entry.getValue();
31                key = entry.getKey();
32            }
33        }
34        for( Map.Entry<Character,Integer> entry : map.entrySet()){
35            System.out.println("[Char :'" +entry.getKey()+"\''+" Used time :'"
36            +entry.getValue()+"']");
37        }
38    }
39 }
```

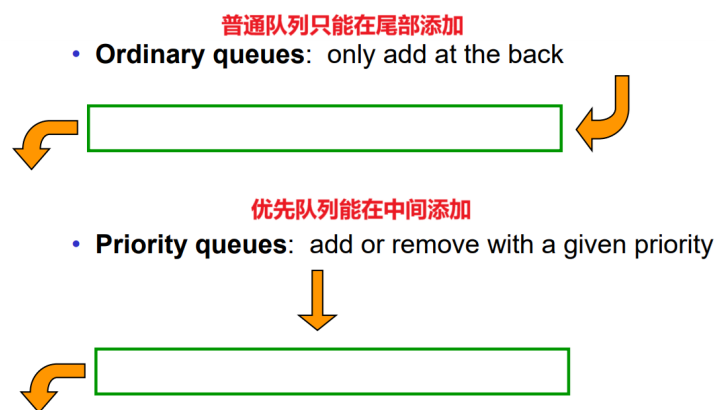
```
1 [Char : 'H' Used time : '1']
2 [Char : 'I' Used time : '2']
3 [Char : 'M' Used time : '5']
4 [Char : 'O' Used time : '2']
5 [Char : 'T' Used time : '1']
6 [Char : ' ' Used time : '1']
7 [Char : ' ' Used time : '77']
8 [Char : 'a' Used time : '36']
9 [Char : 'b' Used time : '1']
10 [Char : 'c' Used time : '8']
11 [Char : 'd' Used time : '20']
12 ...
```

■ [Week 4 Lecture 5]

[1] 队列

队列与栈相似但是不同

1. 队列是具有**顺序**的值的集合
2. 受限的访问：只能从前面移除
3. 两种类型：
 1. 普通队列：只能在尾部添加
 2. 优先队列：根据给定的优先级添加或移除



用途

- 操作系统、网络应用、多用户系统
- 处理必须按顺序完成的请求/事件/任务（在这种上下文中，通常将存储此类请求的内存池称为“缓冲区”）
- 模拟程序
- 表示现实世界中的队列（交通、客户、送货等）
- 管理将来必须发生的事件
- 搜索算法
- 计算机游戏
- 人工智能
- Java提供：一个 `Queue` 接口，多个类： `LinkedList`、`PriorityQueue`

队列操作

- `offer(value) ⇒ boolean`
 - 将值添加到队列中
 - （有时称为“入队”）
- `poll() ⇒ value`
 - 移除并返回队列前端/头部的值，如果队列为空则返回 `null`
 - （有时称为“出队”，类似于“弹出”）
- `peek() ⇒ value`
 - 返回队列头部的值，如果队列为空则返回 `null`

(不会从队列中移除)

- `remove()` 和 `element()`
 - 类似于 `poll()` 和 `peek()`，但如果队列为空则会抛出异常。

使用标准 for-each 循环遍历

```
1  for (Face face : crowd) {
2      face.render(canvas);
3  }
4  for (Map.Entry<String,Integer> entry : phonebook.entrySet()){
5      textArea.append(entry.getKey()+" : "+entry.getValue());
6  }
```

为什么需要迭代器？

- 程序无法直接访问集合 `Collection` 对象的内部
- 集合对象会构造一个迭代器
- 迭代器可以访问集合的内部
- 迭代器逐个提供元素
- 每个集合类都需要一个关联的迭代器类

[2] 迭代器

1. 迭代器和可迭代对象的基础概念

- `Iterable` 接口要求实现类提供一个 `Iterator`。
- `Iterator` 接口要求实现类提供 `hasNext()` 和 `next()` 方法。
- 例如，所有集合（`Collections`）都实现了 `Iterable` 接口。

2. 迭代器的使用

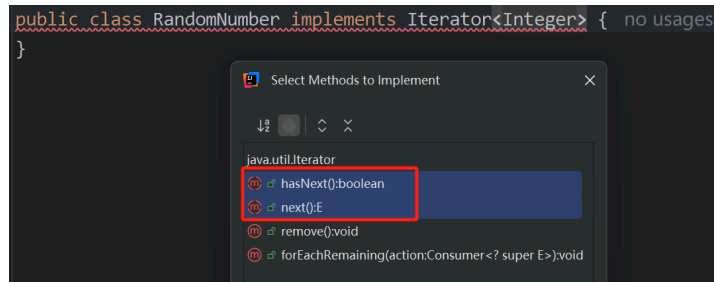
- 迭代器的操作：
 - `hasNext()`：如果还有值可以获取，则返回 `true`
 - `next()`：返回下一个值
- 标准使用模式：

```
1  Iterator<type> itr = construct iterator; // 构造迭代器
2  while (itr.hasNext()) {
3      type var = itr.next(); // 获取下一个值
4      ... var ... // 使用该值
5  }
```

- 几乎与“for each”循环相同：

```
1  for (type var : collection) {
2      ... var ... // 使用集合中的每个值
3  }
```

3. 创建迭代器



要实现 `Iterator<E>` 方法必须 `@Override` 这两个方法

1. `hasNext()`，返回Boolean
2. `next()`，返回泛型 `E`

在下面这个例子中，我们把整型当成 `E`。

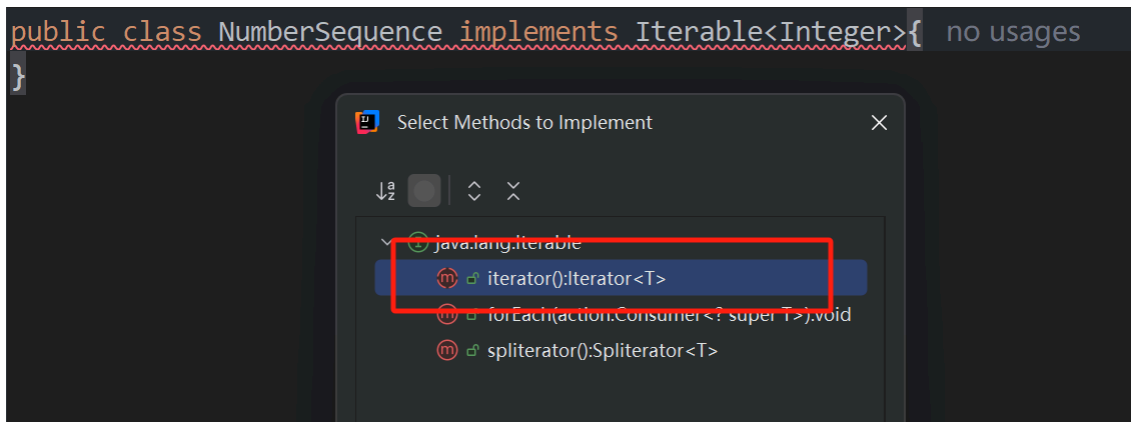
- 迭代器不仅用于集合对象。
- 可以用于任何生成值序列的对象，例如 `Scanner` 或伪随机数生成器。
- 伪随机数生成器的示例：

```
1 public class NumCreator implements Iterator<Integer> {
2     private int num = 1;
3     public boolean hasNext() {
4         return true;
5     }
6     public Integer next() {
7         num = (num * 92863) % 104729 + 1;
8         return num;
9     }
10 }
```

- 使用示例：

```
1 Iterator<Integer> lottery = new NumCreator();
2 for (int i = 1; i < 1000; i++)
3     textArea.append(lottery.next() + "\n");
```

4. 创建可迭代对象



要实现 `Iterable<E>` 方法必须 `@Override` 一个方法

1. `iterator()`，返回一个迭代器对象 `Iterator<E>`

- 例如：`NumberSequence` 类表示一个无限的算术序列，包含起始数字和步长，例如 5, 8, 11, 14, 17,....

```
1  ``java
2  public class NumberSequence implements Iterable<Integer> {
3      private int start;
4      private int step;
5      public NumberSequence(int start, int step) { // 构造方法
6          this.start = start;
7          this.step = step;
8      }
9      public Iterator<Integer> iterator() { // 返回Iterator<Integer>型迭代器对象
10
11          // 传入 NumberSequence 这个可迭代对象
12          return new NumberSequenceIterator(this);
13      }
14  }
15  ``
```

接下来我们来创建 `NumberSequenceIterator`

5. 为可迭代对象 `NumberSequence` 创建迭代器

- `NumberSequenceIterator` 类实现了 `Iterator` 接口。

```
1  // 可迭代对象 NumberSequence
2  public class NumberSequence implements Iterable<Integer> {
3      private int start;
4      private int step;
5      public NumberSequence(int start, int step) { // 构造方法
6          this.start = start;
7          this.step = step;
8      }
9      public Iterator<Integer> iterator() {
10         return new NumberSequenceIterator(this);
11     }
12     // 内部类，迭代器NumberSequenceIterator
13     private class NumberSequenceIterator implements Iterator<Integer> {
14         private int nextNum;
15         private NumberSequence source;
16         public NumberSequenceIterator(NumberSequence ns) {
17             source = ns; // 赋值，获取到NumberSequence这个可迭代对象
18             nextNum = ns.start;
19         }
20         public boolean hasNext() {
21             return true;
22         }
23         public Integer next() {
24             int ans = nextNum;
25             nextNum += source.step;
26             return ans;
27         }
28     }
29 }
```

```
28     }
29 }
```

6. 使用可迭代对象

- 可以在“for each”循环中使用可迭代对象：

```
1 for (int n : new NumberSequence(15, 8)) {
2     System.out.printf("next number is %d \n", n);
3 }
```

- 也可以直接使用可迭代对象的迭代器：

```
1 Iterator<Integer> iter = new NumberSequence(15, 8).iterator();
2 processFirstPage(iter);
3 for (int p = 2; p < maxPages; p++)
4     processNextPage(p, iter);
```

总结：这段内容展示了如何使用迭代器和可迭代对象来处理序列数据，并提供了具体的代码示例来演示如何创建和使用它们。

[3] Q&A

- Java has specified a “Queue” interface. (T or F)

- Java 是否定义了 “Queue” 接口。 (T 或 F)

答案：T

Java 确实定义了 `Queue` 接口。

- Java does not have any class support for “Priority Queue”. (T or F)

- Java 是否没有提供任何支持 “Priority Queue” 的类。 (T 或 F)

答案：F

Java 提供了 `PriorityQueue` 类来支持优先队列。

- peek() operation under the Queue interface will throw an exception if the queue is empty. (T or F)

- 如果队列为空，`Queue` 接口中的 `peek()` 操作是否会抛出异常。 (T 或 F)

答案：F

`peek()` 操作在队列为空时返回 `null`，不会抛出异常。

- poll() operation under the Queue interface will throw an exception if the queue is empty. (T or F)

- 如果队列为空，`Queue` 接口中的 `poll()` 操作是否会抛出异常。 (T 或 F)

答案：F

`poll()` 操作在队列为空时返回 `null`，不会抛出异常。

- There is an element() method under the Queue interface. (T or F)

- `Queue` 接口中是否有一个 `element()` 方法。 (T 或 F)

答案：T

`Queue` 接口确实定义了 `element()` 方法，它在队列为空时会抛出 `NoSuchElementException`。

- Iterable is an interface specification for a class that is equipped with an Iterator.

- `Iterable` 是一个接口规范，用于具有迭代器的类。

答案：T

`Iterable` 接口要求实现类提供一个 `Iterator`。

- Iterator is an interface specification for a class that can generate iterative elements.

- `Iterator` 是一个接口规范，用于可以生成迭代元素的类。

答案：T

`Iterator` 接口要求实现类提供 `hasNext()` 和 `next()` 方法，用于生成迭代元素。

■ [Week 4 Lecture 6]

排序集合

[1] 两种排序方式

第一种方法，默认排序/自然排序

让 `Student` 类实现 `Comparable` 接口。

```
1 public class student implements Comparable<student>{
2     @Override
3     public int compareTo(student o) {
4         return this.age - o.age;
5     }
6 }
```

第二种方法：创建比较器对象（匿名内部类）

```
1 TreeSet<String> ts = new TreeSet<>(new Comparator<String>() {
2     @Override
3     public int compare(String o1, String o2) {
4         if(o1.length() > o2.length()) return 1;
5         return o2.compareTo(o1);
6     }
7 });
```

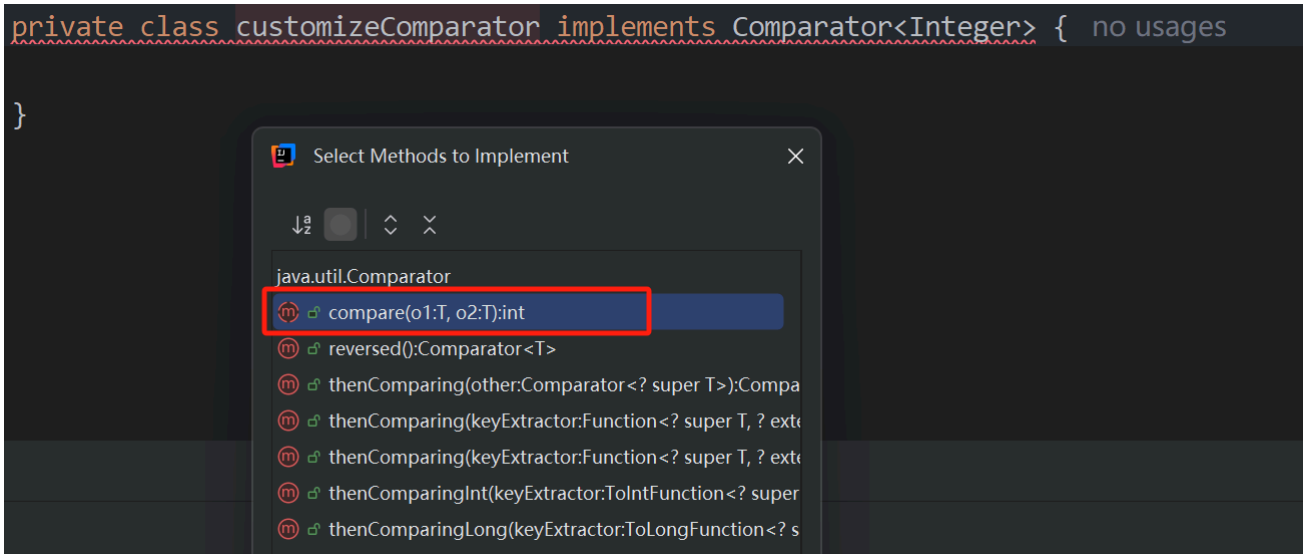
如果两种方法同时存在，以方式二为准（例如 `String` 下定义了比较方法，但是仍然可以用比较器 `Comparator` 来比较）。

例子：

```
1 public static void main(String[] args) {
2     int[] arr1 = {2, 1, 4, 3, 5};
3     // 升序排列（默认）
4     Arrays.sort(arr1);
5     System.out.println(Arrays.toString(arr1));
6     // 降序排列
7     Integer[] arr2 = {2, 1, 4, 3, 5};
8     Arrays.sort(arr2, new Comparator<Integer>() {
9         @Override
10        public int compare(Integer o1, Integer o2) {
11            return o2 - o1;
12        }
13    });
14    System.out.println(Arrays.toString(arr2));
15    // 降序排列（用Lambda表达式简化）
16    arr2 = new Integer[]{2, 1, 4, 3, 5};
17    Arrays.sort(arr2, (Integer o1, Integer o2) -> {
18        return o2-o1;
19    });
19 }
```

```
20 System.out.println(Arrays.toString(arr2));
21 }
```

除了在类中定义，或者使用匿名内部类的方式，我们还可以自己创建一个Compare类（通过实现Compare接口）
要实现这个类必须重写 `compare` 方法



```
1 public static void main(String[] args) {
2     ArrayList<Integer> list = new ArrayList<Integer>();
3     list.add(3);
4     list.add(1);
5     list.add(4);
6     list.add(5);
7     list.add(2);
8
9     System.out.println(list);
10    Collections.sort(list, new customizeComparatorIncrease());
11    System.out.println(list);
12    Collections.sort(list, new customizeComparatorDecrease());
13    System.out.println(list);
14 }
15 private static class customizeComparatorIncrease implements Comparator<Integer> {
16
17     @Override
18     public int compare(Integer o1, Integer o2) {
19         return o2-o1; // 降序
20     }
21 }
22 private static class customizeComparatorDecrease implements Comparator<Integer> {
23
24     @Override
25     public int compare(Integer o1, Integer o2) {
26         return o1-o2; // 降序
27     }
28 }
```

输出如下

```
1 [3, 1, 4, 5, 2]
2 [5, 4, 3, 2, 1]
3 [1, 2, 3, 4, 5]
```

确实可以通过自定义比较器来进行比较。

`Comparable` 和 `Comparator` 接口的区别和用法：

1. `Comparable` 接口：

- 类应该实现 `Comparable` 接口来控制它们的自然排序。
- 实现 `Comparable` 的对象可以通过 `Collections.sort()` 和 `Arrays.sort()` 进行排序，并且可以作为排序映射中的键或排序集合中的元素，而无需指定 `Comparator`。
- `compareTo(Object)` 方法用于将此对象与另一个对象进行比较，返回负整数、零或正整数，分别表示此对象小于、等于或大于另一个对象。

2. `Comparator` 接口：

- 使用 `Comparator` 可以按照非自然顺序对对象进行排序。
- `compare(Object, Object)` 方法用于比较两个参数的顺序，返回负整数、零或正整数，分别表示第一个参数小于、等于或大于第二个参数。

总的来说，`Comparable` 用于定义对象的自然排序，而 `Comparator` 用于定义自定义的排序规则。

[2] Q&A

1. An object defined under a comparable class will have a “natural ordering”. (T or F)

翻译：一个定义在 `Comparable` 类下的对象将会具有“自然排序”。（正确或错误）

True

解释：一个类实现 `Comparable` 接口后，它的对象就具有“自然排序”，因为 `Comparable` 接口定义了 `compareTo` 方法。

2. Objects declared under a comparable class can be compared using which method?

翻译：声明在 `Comparable` 类下的对象可以使用哪个方法进行比较？

`compareTo` 方法

解释：`Comparable` 接口中的 `compareTo` 方法用于比较对象。

3. What is the signature of the compareTo method?

翻译：`compareTo` 方法的签名是什么？

`int compareTo(T o)`

解释：`compareTo` 方法的签名为 `int compareTo(T o)`，其中 `T` 是泛型类型。

4. Which method can be used to sort list of comparable objects?

翻译：哪个方法可以用来排序 `Comparable` 对象的列表？

`Collections.sort()`

解释：`Collections.sort()` 方法可以用来排序实现 `Comparable` 接口的对象的列表。

5. Comparator is an object that can compare other objects. (T or F)

翻译： `Comparator` 是一个可以比较其他对象的对象。（正确或错误）

True

解释： `Comparator` 是一个接口，它可以用于比较两个对象，因此它是可以比较其他对象的对象。

6. What is the signature for the compare() method?

翻译： `compare()` 方法的签名是什么？

`int compare(T o1, T o2)`

解释： `compare()` 方法的签名为 `int compare(T o1, T o2)`，其中 `T` 是泛型类型。

7. A comparable class can implement multiple comparators. (T or F)

翻译： 一个 `Comparable` 类可以实现多个 `Comparator`。（正确或错误）

True

解释： 一个 `Comparable` 类可以实现多个 `Comparator`，因为 `Comparator` 是独立的接口，可以定义多种比较逻辑。

■ [Week 4 Lecture 7]

异常 Exception

[1] 异常处理（Exceptions）

这段内容主要讲解了异常的概念、抛出异常的原因、异常的类型以及如何在代码中捕获和处理异常。

1. 什么是异常？

- 异常是程序执行过程中发生的事件，它会中断程序的正常指令流。
- 异常是一种“非局部退出”，这意味着它会打断当前的执行流程并跳转到异常处理逻辑。

2. 何时会抛出异常？

- 发生某种错误时，例如：
 - 文件不存在
 - 除以零
 - 在空对象上调用方法
 - 调用未定义的方法
- 出现异常情况时，例如：
 - 文件中的文本不符合预期格式
 - 用户输入了意外的答案

3. 如何处理异常？

- 抛出异常（`throw`）
- 捕获异常（`catch`）

4. 异常的来源

- JVM 在执行代码时抛出
- Java 库中的代码抛出
- 开发者的代码主动抛出

5. 捕获异常

使用 `try...catch` 语句捕获异常

例如，我们写了一个能够写入文件的 `方法`：


```

1 public static void write(String path, String text,int mode){
2     boolean append = false;
3     if(mode == OVERRIDE_ALL) append = false;
4     if(mode == APPEND) append = true;
5
6     try {
7         writer = new BufferedWriter(new FileWriter(path,append));
8         writer.write(text);
9         writer.close();
10    } catch (IOException e) {
11        throw new RuntimeException(e);
12    }
13 }

```

在这里就有捕获 `IOException` 的 `try-catch` 方法了

异常类型

- `RuntimeException` 不需要处理：
 - 未捕获的 `RuntimeException` 会导致错误信息
 - 如果你愿意，可以捕获它们。
- 其他异常必须处理：
 - 例如 `IOException`
(这就是为什么我们在打开文件时总是使用 `try...catch`)。
- 异常对象包含信息：
 - 执行栈的状态
 - 任何与异常相关的额外信息

[2] Q&A

1. A method will do what when something goes wrong?

翻译：当出现问题时，方法会做什么？

答案：方法会抛出异常（`throw an exception`）。

2. An exception provides a local exit when something goes wrong. (T or F)

翻译：异常在出现问题时提供了一种局部退出的方式。（正确或错误）

True

解释：异常是一种机制，可以在出现错误时立即停止当前代码的执行，并跳转到异常处理代码。

3. Name 3 cases when an exception can be thrown.

翻译：列举3种可能抛出异常的情况。

答案：

- 访问数组时下标越界（`ArrayIndexOutOfBoundsException`）。
- 除零操作（`ArithmeticException`）。
- 访问空对象的方法或属性（`NullPointerException`）。

4. What do we do with exceptions?

翻译：我们如何处理异常？

答案：我们可以捕获异常（`catch`）并处理它，或者将异常抛出（`throw`）给调用者。

5. Exceptions can be caught using what Java statement?

翻译：异常可以使用哪种Java语句捕获？

答案：使用 `try-catch` 语句。

6. Does exception have a type?

翻译：异常是否有类型？

答案：是，异常是 `Throwable` 的子类，例如 `Exception` 和 `RuntimeException`。

7. After the exception handler finishes its work, what will the program do next?

翻译：异常处理程序完成工作后，程序接下来会做什么？

答案：程序会继续执行 `try-catch` 块之后的代码。

8. Can exception handler use information in the exception object?

翻译：异常处理程序可以使用异常对象中的信息吗？

答案：是，例如通过 `getMessage()` 方法获取异常的详细信息。

9. What does “e.getMessage()” do where e is an exception object?

翻译：其中 `e` 是异常对象，“e.getMessage()”会做什么？

答案：返回异常的详细信息字符串。

10. Name 3 common Java exceptions.

翻译：列举3种常见的Java异常。

答案：

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `IOException`

11. `RuntimeException` doesn't have to be handled. (T or F)

翻译：`RuntimeException` 不需要被处理。（正确或错误）

True

解释：`RuntimeException` 是 `unchecked exception`，不强制要求捕获或声明。

12. `IOException` doesn't have to be handled. (T or F)

翻译：`IOException` 不需要被处理。（正确或错误）

False

解释：`IOException` 是 `checked exception`，必须捕获或声明抛出。

13. Which Java statement can cause an exception?

翻译：哪种Java语句可能导致异常？

答案：`throw` 语句。

14. What happens when we try adding an element to an immutable List?

翻译：当我们尝试向一个不可变的 `List` 添加元素时会发生什么？

答案：抛出 `UnsupportedOperationException`。

15. Does a Java Interface provide a 'constructor'?

翻译：Java接口是否提供“构造函数”？

答案：否，接口不能包含构造函数。

16. **ArrayList implements which Interface?**

翻译: `ArrayList` 实现了哪个接口?

答案: `List`。

17. **public interface List extends which Java Interface?**

翻译: `public interface List <E>` 继承自哪个Java接口?

答案: `Collection<E>`。

■ [Week 5 Lecture 8,9,10]

实现Collections: Interfaces, Abstract Classes, Classes

介绍了接口和继承（抽象）类

Lecture 8

[1] ArrayList 设计思路

`List` 是 Java 中的一个接口，它定义了很多方法，比如添加元素、获取元素、删除元素、判断是否包含某个元素等等。`ArrayList` 是实现 `List` 接口的一个类，这就意味着 `ArrayList` 必须实现 `List` 接口中定义的所有方法。但是 `List` 接口中的方法数量很多，而且其中不少方法的实现逻辑比较复杂，这给 `ArrayList` 类的编写带来了困难。

解决思路：

虽然 `List` 接口中的方法很多，但其实许多复杂方法可以通过几个基本的方法来实现。这里提到的基本方法包括 `size`（用于获取集合中元素的数量）、`add`（用于向集合中添加一个元素）、`get`（用于根据索引获取集合中的元素）、`set`（用于根据索引设置集合中某个位置的元素）、`remove`（用于移除集合中的某个元素）。

先看构造：

```
1 public class ArrayList <E> implements List<E> {
2     private E [ ] data; // 泛型数组数据
3     private int count; // 记录数量
4     ....
5 }
```

虽然有些方法很复杂，例如 `addAll()`，但是可以通过调用自身的**基本方法**来实现。例如调用基本方法 `add()`：

```
1 public boolean addAll(Collection<E> other){
2     for (E item : other) // 增强for遍历other然后利用this.add()这个自身方法添加...
3         add(item);
4 }
```

实际上的ArrayList：

```
1 public class ArrayList<E> extends AbstractList<E>
2     implements List<E>, RandomAccess, Cloneable, java.io.Serializable
3 {
4     ...
5 }
```

先看继承的抽象类

抽象类里既可以有抽象方法（无方法体），也可以有普通方法（有方法体）。

可以看到`ArrayList`继承了`AbstractList`这个抽象类

然后`AbstractList`里面就写了一些简单的方法（有方法体），当然也有抽象方法（无方法体）。

为何要让`AbstractList`设计为抽象类？因为它需要强制（继承他的）子类复写所有抽象方法，不然自身的一些方法就不可用，例如

```
1 // AbstractList
2 public abstract E get(int index);
3 public abstract int size();
4 // 无方法体
```

然而，在`AbstractList`中，有一些方法依赖`get()`方法，例如`Iterator()`

```
1 // AbstractList
2 public Iterator<E> iterator() {
3     return new Itr();
4 }
```

```
1 // AbstractList
2 private class Itr implements Iterator<E> {
3     ...
4
5     public boolean hasNext() {
6         return cursor != size(); // 这里用到了
7     }
8
9     public E next() {
10        ...
11        try {
12            ...
13            E next = get(i); // 这里用到了
14            ...
15        } catch (IndexOutOfBoundsException e) {
16            ...
17        }
18    }
19 }
20 }
```

再看继承的接口

接口中的方法都是抽象方法，没有方法体。在 Java 8 之前，接口只能包含抽象方法；Java 8 及以后，接口可以包含默认方法和静态方法，它们有具体的方法体。

一个类可以实现多个接口，这打破了类的单继承限制，让类可以具备多种不同的行为。

接口里的变量默认是 `public static final` 类型的，也就是常量，必须在定义时就进行初始化，并且之后不能再修改。

接口的抽象方法和默认方法

```
1 interface Vehicle {
2     // 抽象方法
3     void start();
4     // 默认方法
5     default void honk() {
6         System.out.println("Vehicle is honking!");
7     }
8 }
9 // 子类不重写 honk 方法，将使用接口中的默认实现
```

接口的静态方法

使用 `static` 关键字修饰，在接口中提供静态的实现。静态方法属于接口本身，不能通过实现类的实例来调用，只能通过接口名直接调用。

```
1 interface MathUtils {
2     // 静态方法，计算两个数的和
3     static int add(int a, int b) {
4         return a + b;
5     }
6 }
```

好了，现在我们已经了解完接口，让我们看看ArrayList继承了什么接口！

```
1 public class ArrayList<E> extends AbstractList<E>
2     implements List<E>, RandomAccess, Cloneable, java.io.Serializable
3 {
4     ...
5 }
```

最明显的时它继承了 `List<E>` 这个接口

```

List
  add(E): boolean ↑Collection
  add(int, E): void
  addAll(Collection<? extends E>): boolean ↑Collection
  addAll(int, Collection<? extends E>): boolean
  addFirst(E): void ↑SequencedCollection
  addLast(E): void ↑SequencedCollection
  clear(): void ↑Collection
  contains(Object): boolean ↑Collection
  containsAll(Collection<?>): boolean ↑Collection
  copyOf(Collection<? extends E>): List<E>
  equals(Object): boolean ↑Collection
  get(int): E
  getFirst(): E ↑SequencedCollection
  getLast(): E ↑SequencedCollection
  hashCode(): int ↑Collection
  indexOf(Object): int
  isEmpty(): boolean ↑Collection
  iterator(): Iterator<E> ↑Collection

```

可以发现实际上List接口都是做了一些规范，然后让实现类去复写一般抽象方法，当然也有static方法，这些方法只能调用接口使用。 -

类别	描述	示例（以Java集合相关为例）
Interface	指定类型；定义方法头	<code>List <E></code> ：指定E类型的序列
Abstract Class	实现接口；定义部分方法，将其他方法设为“抽象”	<code>AbstractList <E></code> ：实现 <code>List <E></code> 接口；定义E类型数组；定义 <code>addAll</code> 、 <code>subList</code> 等方法； <code>add</code> 、 <code>set</code> 、 <code>get</code> 等方法为抽象方法
Class	继承抽象类；定义数据结构、基本方法和构造函数	<code>ArrayList <E></code> ：继承 <code>AbstractList</code> ；实现字段和构造函数；实现 <code>add</code> 、 <code>get</code> 等方法

关于泛型数组

泛型数组不允许直接创建。比如我们现在想要创建一个 `E[]`

```

public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    public MyList() { no usages new *
        data = new E[size];
    }
}

```

这是不允许的。但是我们可以创建一个 `Object[]` 然后再用类型转换转为 `E[]`

```

public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    public MyList() { no usages new *
        data = (E[]) new Object[size];
    }
}

```

Unchecked cast: 'java.lang.Object[]' to 'E[]' :
© java.lang.Object

此时编译器会弹出警告：未检查的映射，为了“隐藏”这个问题，可以加上：

```

public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    @SuppressWarnings("unchecked") no usages new *
    public MyList() {
        data = (E[]) new Object[size];
    }
}

```

© java.lang.Object
@Contract(pure = true) ↗
public Object()

[2] ArrayList中的一些简单方法

set方法

```

1 public E set(int index, E value){
2     if (index < 0 || index >= count)
3         throw new IndexOutOfBoundsException();
4     E ans = data[index];
5     data[index] = value;
6     return ans; // ans时弹出值,就是旧值。如果没有则是默认值
7 }

```

remove方法：

```

1 public E remove (int index){
2     if (index < 0 || index >= count)
3         throw new IndexOutOfBoundsException();
4     E ans = data[index]; // 旧值
5     for (int i=index+1; i< count; i++) // 这里就体现了ArrayList的删除过程巨慢
6         data[i-1]=data[i];
7     count--; // 数量-1
8     return ans;
9 }

```

注意！现在我们要删除index，那么下面哪种删除方法是正确的？


```

1  for (int i=index; i < count; i++)
2      data[i]=data[i+1];

```

```

1  for (int i=index+1; i < count; i++)
2      data[i-1]=data[i];

```

第一种会有数组越界风险，因为 `i==count-1` 没问题但是 `i+1 == count` 可能导致数组越界

add 方法

```

1  public void add(int index, E item){
2      if (index < 0 || index > count)
3          throw new IndexOutOfBoundsException(); // 如果用户输入的index大于count或者小于0那么就
    报错
4      ensureCapacity();// 这一步很重要，确保容量足够
5      for (int i=count; i > index; i--) // 这种方法规避了数组越界风险
6          data[i]=data[i-1];
7      data[index]=item;
8      count++;
9  }

```

ensureCapacity 方法

```

1  private void ensureCapacity () {
2      if (count < data.length) return; // 空间足够直接返回
3      E [ ] newArray = (E[]) (new Object[data.length+INITIALCAPACITY]); // 第一种：直接加
    INITIALCAPACITY
4      for (int i = 0; i < count; i++) // 复制到newArray
5          newArray[i] = data[i];
6      data = newArray; // 成员变量data现在储存newArray地址值，然后旧的data的地址值因为没有指向而被回
    收
7  }

```

```

1  // 第二种是
2  E [ ] newArray = (E[ ]) (new Object[data.length * 2]);

```

[3] 课后小问题

[问题 1] What are the key features of an abstract class?

翻译：抽象类的关键特征是什么？

1. 抽象类可以包含抽象方法（abstract methods），这些方法只有声明而没有实现。例如：

```

1  abstract class Shape {
2      // 抽象方法
3      public abstract double getArea();
4  }

```

2. 抽象类不能被实例化，只能被继承。例如，上述 `Shape` 类不能使用 `new Shape()` 来创建对象。
3. 抽象类可以包含具体的方法和成员变量，就像普通类一样。例如：

```
1 abstract class Shape {  
2     protected String color; // 成员变量  
3     public Shape(String color) { // 具体方法，子类可选择性重写  
4         this.color = color;  
5     }  
6     public abstract double getArea(); // 子类必须重写  
7 }
```

在 Java 里，`protected` 成员（包括字段与方法）能够在以下范围内被访问：

1. **同一类内部**：在定义了 `protected` 成员的类内部，该成员可被自由访问。
2. **同一包内**：处于同一包中的其他类，不管这些类是否为该类的子类，都可以访问 `protected` 成员。
3. **不同包的子类**：在不同包中的子类能够访问从父类继承来的 `protected` 成员。不过，子类对象只能通过自身对象来访问这些成员，而不能通过父类对象访问。

[问题 2] **Can an abstract class be instantiated?**

翻译：抽象类可以被实例化吗？

答案：不可以。但是我们可以使用匿名内部类的方式直接创建对应的类，因此来创建对象。

匿名内部类可以直接实现接口和抽象类。

[问题 3] **Abstract methods can be defined within a class to save implementation efforts. (T or F)**

翻译：可以在类中定义抽象方法以节省实现工作。（对或错）

答案：True。

就像我们上面讲的，`ArrayList`就实现了 `AbstractList` 这个抽象类，而且这个抽象类实现了很多方法，比如 `Iterator()`。但是同时有必须要求子类重写 `get()` 和 `size()` 等抽象方法，这样 `AbstractList` 才能正常工作。

[问题 4] **What are the key issues of implementation when we remove an element from an ArrayList?**

翻译：当从 `ArrayList` 中移除一个元素时，实现的关键问题有哪些？

答案：小心数组越界。然后记得返回旧值。

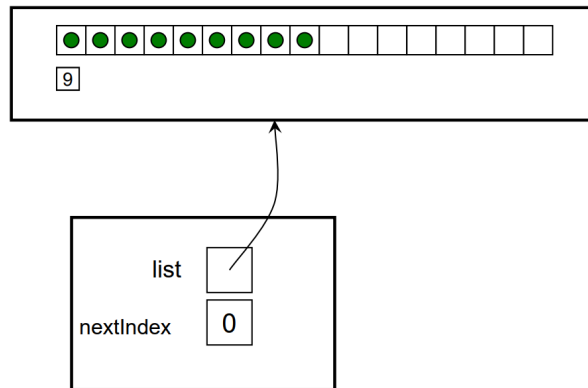
[问题 5] What are the key issues of implementation when we add an element from an ArrayList?

翻译：当向 ArrayList 中添加一个元素时，实现的关键问题有哪些？

答案：检查扩容，大小调整。

Lecture 9

[4] 迭代器 Iterator



基本构造原理

一个对外的类，`Iterator`，当被调用，返回 `ArrayListIterator`。

`ArrayListIterator` 实在 `ArrayList` 中的内部类，因此可以访问 `ArrayList` 的私有成员变量

```
1  /**
2   * 返回一个用于遍历列表中元素的迭代器
3   */
4  public Iterator <E> iterator() {
5      return new ArrayListIterator<E>(this); // 这里传入 arraylist对象
6  }
7
8  /**
9   * 定义用于 ArrayList 的迭代器
10  * 该迭代器定义在 ArrayList 类内部，因此可以访问 ArrayList 对象的私有成员变量
11  */
12 private class ArrayListIterator <E> implements Iterator <E> {
13     // 成员变量
14     // 构造方法
15     // hasNext() 方法
16     // next() 方法
17     // remove() 方法（迭代器的可选操作）
18 }
```

现在，让我们看看 `ArrayListIterator` 到底有什么吧！首先，我们简单看看他是如何能够访问 `ArrayList` 的：

```
1  // ArrayListIterator 实现了 Iterator接口
2  // 因为ArrayListIterator是定义在ArrayList内部的类，因此可以访问ArrayList的成员变量
```

```

3 private class ArrayListIterator <E> implements Iterator <E>{
4
5     private ArrayList<E> list; // 这个就是待迭代的list
6     private int nextIndex = 0; // 指向下一个index
7     private boolean canRemove = false; // 能否remove
8     private ArrayListIterator (ArrayList <E> list) { // 这里传入 arrayList对象
9         this.list = list;
10    }
11
12    // 是否有下一个元素
13    public boolean hasNext () {
14        return (nextIndex < list.count);
15    }
16    ...
17 }

```

然后，我们再看看它的方法...

next 和 remove方法

```

1 // 返回下一个元素
2 public E next () {
3     if (nextIndex >= list.count) throw new NoSuchElementException();
4     return list.get(nextIndex++); // 比如现在我们第一次获取
5     // 首先，会得到 ArrayList[0]
6     // 接下来nextIndex = 1
7     // return ArrayList[0]而不是1 !!!
8 }
9
10
11 // 该方法的功能是从列表里移除迭代器最后返回的那个元素。
12 public void remove(){
13     throw new UnsupportedOperationException();
14 }
15
16 }

```

这个remove需要更详细地说，就是你只能调用1次（每次next前）

```

1 while (iterator.hasNext()) {
2     Integer number = iterator.next(); // 好，现在迭代器返回了这个元素
3     if (number % 2 == 0) {
4         iterator.remove(); // 这没问题，你删除了现在对应的元素
5     }
6 }

```

```

1 while (iterator.hasNext()) {
2     Integer number = iterator.next();
3     if (number % 2 == 0) {
4         iterator.remove();
5         iterator.remove(); // 这里有问题！你多删除了一次！迭代器根本不知道你要删除谁，因为
    前一个已删除！
6     }
7 }

```

而且，使用迭代器的时候不允许使用 `number.remove()`，因为会带来并发修改异常！（之前讲过）

更详细的remove方法

接下来，我们来讲下他们两是怎么实现的

```

1 /** Return next element in the List */
2 public E next () {
3     if (nextIndex >= list.count) throw new NoSuchElementException();
4     canRemove = true; // 能删除
5     return list.get(nextIndex++); ← increment and return
6 }
7
8 public void remove(){
9     if ( ! canRemove ) throw new IllegalStateException();
10    canRemove = false;
11    nextIndex--; // 这个很重要
12    list.remove(nextIndex);
13 }

```

关于 `nextIndex--`

假设我们现在的ArrayList如下：

Index	0	1	2	3	4
Value	110	123	723	26	10

现在我们用 `Iterator` 来迭代，目标是删除2

现在 `next()` 执行，`canRemove = True`，然后return了723，接下来 `nextIndex++`，变成了3。

但是我们next完之后想要remove 723,因此调用 `remove` 方法，此时 `index --`，变成了2。成功删除了3

Index	0	1		2	3
Value	110	123		26	10

这是正确的，因为指向了下一个元素 26

再再注意，由于迭代器是一个内部类，它可以访问 `ArrayList` 的私有成员变量（字段）。

[5] 课后小问题

[问题 1] `remove()` is compulsory in Iterator implementation. (T or F)

翻译：在迭代器（Iterator）实现中，`remove()` 方法是必须的。（对或错）

答案：False。虽然 `Iterator` 接口定义了 `remove()` 方法，但在具体实现迭代器时，该方法不是强制必须实现的。如果迭代器不支持移除操作，可以在 `remove()` 方法中抛出 `UnsupportedOperationException` 异常。例如，在某些只读的迭代器实现中，就不需要实现实际的移除逻辑，只抛出异常即可。

[问题 2] How does ArrayList make use of the 'type parameter' in its implementation?

翻译：ArrayList 在其实现中是如何使用“类型参数”的？

在 Java 中，`ArrayList` 是一个泛型类，使用类型参数 `<E>` 来指定其存储元素的类型。

1. **类型安全**：在编译时确存储存在 `ArrayList` 中的元素类型一致。例如，`ArrayList<String>` 只能存储 `String` 类型的对象，如果尝试添加其他类型对象，编译器会报错。
2. **代码复用**：通过类型参数，`ArrayList` 可以适应不同类型的元素存储需求，而不需要为每种元素类型编写单独的类。例如，既可以创建 `ArrayList<Integer>` 存储整数，也可以创建 `ArrayList<Double>` 存储双精度浮点数。
3. **自动装箱和拆箱**：当 `ArrayList` 存储基本数据类型的包装类时，会涉及自动装箱和拆箱。例如，`ArrayList<Integer>` 可以直接添加 `int` 类型数据，编译器会自动将 `int` 装箱为 `Integer`。

[问题 3] Which element will be removed by ArrayList.remove()?

翻译：ArrayList.remove() 会移除哪个元素？

`ArrayList` 有两个重载的 `remove()` 方法：

1. `remove(int index)`：移除指定索引位置的元素。
2. `remove(Object o)`：移除列表中首次出现的指定元素。例如

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("apple");
3 list.add("banana");
4 list.add("apple");
5 list.remove("apple");
```

会移除第一个出现的 "apple" 元素。如果元素不存在，该方法返回 `false`；如果成功移除元素，返回 `true`。

[问题 4] What does ArrayList.next() check before returning the next element in the list?

翻译：ArrayList.next() 在返回列表中的下一个元素之前会检查什么？

`ArrayList` 本身没有 `next()` 方法，`next()` 方法是 `Iterator` 接口中的方法，`ArrayList` 返回的迭代器实现了这个方法。在 `Iterator` 的 `next()` 方法返回下一个元素之前，通常会检查以下内容：

是否还有下一个元素：通过调用 `hasNext()` 方法来判断是否还有下一个元素。`hasNext()` 方法会维护一个内部的游标 (cursor)，判断游标是否已经到达列表末尾。如果 `hasNext()` 返回 `false`，调用 `next()` 方法会抛出 `NoSuchElementException` 异常。例如：

[问题 5] **How does `ArrayList.remove()` ensure only 1 element can be removed after each call to `next()`?**

翻译：`ArrayList.remove()` 如何确保每次调用 `next()` 之后只能移除一个元素？

答案：基于内部类迭代器实现，维护一个变量 `canRemove` 来实现

[问题 6] **What can happen if 2 or more Iterators running concurrently under the same `ArrayList`? Name 2 scenarios.**

翻译：如果在同一个 `ArrayList` 下同时运行两个或更多的迭代器，会发生什么？列举两种情况。

1. **数据不一致**
2. **抛出 `ConcurrentModificationException`：**Java 的 `ArrayList` 迭代器采用快速失败 (fail-fast) 机制。当一个迭代器检测到 `ArrayList` 在迭代过程中被结构修改（如添加、移除元素，除了通过迭代器自身的 `remove()` 方法），会抛出 `ConcurrentModificationException` 异常。例如，一个迭代器正在遍历，另一个线程通过 `ArrayList` 的 `add()` 方法添加元素，就可能触发这个异常。

Lecture 10

[6] 分析 Cost

我们如何分析一个 程序、算法 的Cost？从 **时间** 和 **空间** 入手。

对于程序，使用BenchMarking（基准测试）

对于算法，使用复杂度分析

基准测试

测量内容

- **实际程序：**针对实际运行的程序进行考察。
- **在真实机器上：**在真实的物理计算机设备上开展测试，而非模拟环境。
- **针对特定输入：**使用特定的输入数据来运行程序。
- **测量耗时**
 - 通过 `System.currentTimeMillis()` 获取从系统时钟开始到当前的时间（以毫秒为单位，返回值类型为 `long`），以此来度量程序从开始运行到结束所花费的时间。
- **测量实际内存使用情况：**监测程序在运行过程中实际占用的内存量。

值得注意的问题：

- **选择合理的输入**：需要谨慎挑选测试数据集。应使用大规模数据集，并且**不要包含用户输入**。
- **其他用户 / 进程干扰**：其他用户运行的程序或系统中其他进程可能影响测试结果，要尽量减少这种影响。可通过多次运行程序并取平均值的方式来降低干扰。
- **使用哪台计算机**：需明确指定所使用计算机的详细信息，因为不同计算机的硬件配置会对测试结果产生显著影响。

复杂度分析

这是针对算法的。

将“步骤数”作为数据规模的函数来衡量。

- **最坏情况**（较容易分析）
- **平均情况**（较难分析）
- **最好情况**（容易分析，但用处不大）

构建步骤数的表达式：

- 代价 = $3.47n^2 - 67n + 53$ 步
- 代价 = $3n \log(n) - 5n + 6$ 步

简化为关于 n 的不同幂次 / 函数的项。

分析：渐近表示法

Asymptotic Notation

我们只关注当规模很大时的成本。

- **舍去低阶项**：（随着 n 增大，这些项变得无关紧要）
比如成本 = $3.47n^2 + \dots$ 步，成本 = $3n \log(n) + \dots$ 步。
- **忽略常数因子**：实际的常数因子取决于硬件。
所以舍去常数因子，得到 **Cost** $\propto n^2 + \dots$ 步，**成本** $\propto n \log(n) + \dots$ 步。
- **“渐近成本”，即“大O”成本**：描述成本如何随输入规模增长。
 - 成本为 $O(1)$ ：固定成本。
 - 成本为 $O(n)$ ：随 n 增长。

大O表示法

一种描述计算机算法效率的表示法：假设时钟频率为100兆赫兹， $N = 1024k = 2^{20}$ 。

- $O(1)$ - 常数时间，10纳秒。
- $O(\log N)$ - 对数时间，200纳秒。
- $O(N)$ - 线性时间，10.5毫秒。

- $O(N \log N)$ - $n \log n$ 时间，210毫秒。
- $O(N^2)$ - 二次时间，3.05小时。
- $O(N^3)$ - 三次时间，365年。
- $O(2^N)$ - 指数时间， 10^{10^5} 年。

如果数据/输入的规模为 n ，其增长情况如下：

$O(1)$ “常数级”：成本与 n 无关

$O(\log(n))$ “对数级”：规模乘以10，成本增加一点（增加 $\log(10)$ ）

$O(n)$ “线性级”：规模乘以10，成本变为10倍

$O(n \log(n))$ “ n 乘以 $\log(n)$ 级”：规模乘以10，成本变为略多于10倍

$O(n^2)$ “平方级”：规模乘以10，成本变为100倍

$O(n^3)$ “立方级”：规模乘以10，成本变为1000倍

...

$O(2^n)$ “指数级”：规模增加1，成本翻倍
 => 你肯定不想运行这种算法！

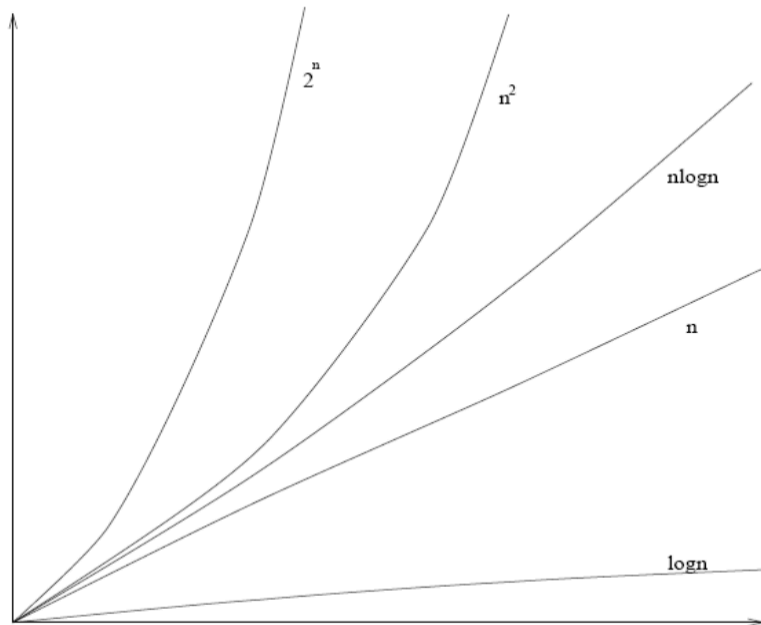
$O(n!)$ “阶乘级”：规模增加1，成本变为 n 倍
 => 你绝对不会想用这种算法！

函数的层次结构

我们可以定义一个函数层次结构，其中每个函数的数量级都比它的前一个函数大：

函数类型	函数示例
常数函数	1
对数函数	$\log n$
根号	$\sqrt{n}$ 比 $\log n$ 增长稍快
多项式函数	$n, n^2, n^3, \dots, n^k$ （还可细化如 $n \log n$, $n^2 \log n$ 等）
指数函数	2^n

这对于CPT104的学习很有帮助！是的，就是104... 这些层级不可逾越，简单来说，举个很极端的例子
 $n^{999999999}$ 总会在某处 $< 2^n$
 note: 阶乘归在了指数层级，但是阶乘还是大于指数



什么是步骤

Step

计数规则

- 最内层循环中的操作（执行次数最多）
- 每次循环都会执行的操作（不在“if”语句内）
- 涉及“数据值”的操作（而不是索引）
- 有代表性的操作（无需统计每一个操作）

```
1 public E remove(int index){
2     if (index < 0 || index >= count) throw new ....Exception();
3     E ans = data[index];
4     for (int i = index + 1; i < count; i++) // 在最内层循环中
5         data[i - 1] = data[i]; // 关键步骤
6     count--;
7     data[count] = null;
8     return ans;
9 }
```

每次 for 循环：

- 1 次比较操作： `i < count`（不计入）
- 1 次加法操作： `i++`（不计入）
- 1 次数据检索操作： `data[i]`（不计入）
- 1 次减法操作： `i - 1`（不计入）
- 1 次内存存储操作： `data[i - 1] = data[i]`（计入，因为这个是开销Cost最大的操作）

那么如何判断、统计开销最大的操作呢？

- 两个数字相加开销较小，但是求幂运算开销相对较大。
- 比较两个字符串可能开销较大。
- 从文件中读取一行内容开销可能**非常大**。
- 等待用户或其他程序的输入可能耗时极长.....

评估ArrayList的方法Method

操作	时间复杂度
get	O(1)
set	O(1)
remove	O(n)
add (at i)	O(n)（最坏和平均情况）
add (at end)	O(1)（平均情况）
add (at end)	O(n)（最坏情况）
add (at end)	O(1)（均摊 amortised 平均情况）

[7] Q&A

[题目1] $O(\log(n)) < O(\sqrt{n})$ (T or F)

翻译： $O(\log(n))$ 小于 $O(\sqrt{n})$ （对或错）

答案：T

解释：随着 n 的增大， $\log(n)$ 的增长速度比 $\sqrt{n}$ 慢。例如，当 n 足够大时， $\log(n)$ 的值会远小于 $\sqrt{n}$ 的值，所以从渐进时间复杂度的角度， $O(\log(n))$ 表示的增长速度小于 $O(\sqrt{n})$ ，该表述正确。

[题目2] $O(n^n) < O(n!)$ (T or F)

翻译： $O(n^n)$ 小于 $O(n!)$ （对或错）

答案：F

解释： $n! = 1 \times 2 \times \dots \times n$ ，而 $n^n = n \times n \times \dots \times n$ 。当 n 增大时， n^n 的增长速度远快于 $n!$ 。可以通过极限等数学方法证明，所以 $O(n^n)$ 大于 $O(n!)$ ，该表述错误。

[题目3] $O(2^n) < O(n^n)$ (T or F)

翻译：O(2^n) 小于 O(n^n)（对或错）

答案：T

解释：随着 n 的增大， 2^n 的增长速度远慢于 n^n 。因为 n^n 中底数和指数都随 n 增大，而 2^n 底数固定为 2。所以 $O(2^n)$ 表示的增长速度小于 $O(n^n)$ ，该表述正确。

[题目4] When analysing the cost of an algorithm, loop usually is the focus. (T or F)

翻译：在分析算法成本时，循环通常是重点。（对或错）

答案：T

解释：循环结构通常会重复执行某些操作，其执行次数往往与输入规模相关，会对算法的时间复杂度产生关键影响。许多算法的时间复杂度主要由循环的执行次数和循环内操作的复杂度决定，所以在分析算法成本时，循环通常是重点，该表述正确。

[题目5] Which of the following operations is more expensive? Reading a line from a file; Reading a line from a user

翻译：以下哪个操作成本更高？从文件中读取一行；从用户读取一行

答案：Reading a line from a user

解释：从用户读取一行涉及到与用户的交互，用户输入的速度不确定，可能会有较长的等待时间，而且可能需要处理用户输入的各种异常情况。而从文件中读取一行相对来说是在计算机存储系统内的操作，速度较为稳定且通常较快。所以从用户读取一行成本更高。

[题目6] Worst case cost analysis is usually more difficult than average cost analysis. (T or F)

翻译：最坏情况成本分析通常比平均成本分析更困难。（对或错）

答案：F

解释：平均成本分析需要考虑所有可能输入的概率分布等因素，要准确评估每个输入情况及其发生概率比较困难。而最坏情况成本分析只需要找出导致算法性能最差的特定输入情况，相对而言，确定最坏情况可能更容易一些。所以通常平均成本分析比最坏情况成本分析更困难，该表述错误。

■ [Week 6 Lecture 11,12,13,14]

[1] 递归

Recursion

目录：

1. 递归函数
2. 阶乘函数
 - Factorial Function
3. 斐波那契数列
4. [递归 Recursion] 和 [迭代 Iteration] 的对比

递归函数

1. 需要有终止条件
2. 要有递归部分
3. 要有处理数据部分

阶乘函数

```
1  #include <stdio.h>
2  long fac(long);
3
4  int main() {
5      int i;
6      for (i = 1; i <= 5; i++)
7          printf("%d! = %ld\n", i, fac(i));
8      return 0;
9  }
10
11 long fac(long number) {
12     // 终止条件
13     if (number <= 1)
14         return 1;
15     // 递归 + 数据处理
16     else
17         return (number * fac(number - 1));
18 }
```

斐波那契数列

$$\text{斐波那契函数可表示为: } f(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

```
1  #include <stdio.h>
2  long long int fibonacci(long long int n);
3
4  int main(){
5      long long int number;
6      scanf("%lld",&number);
7      printf("%lld\n",fibonacci(number));
8  }
9  long long int fibonacci(long long int n){
10     // 终止条件
11     if(n == 1|| n==2) return 1;
12     // 递归+数据处理
13     return fibonacci(n-1) + fibonacci(n-2);
14 }
```

递归和迭代的对比

比较项	递归（Recursion）	迭代（Iteration）
常用结构	调用（自身）函数	while、for 等迭代结构
终止条件	识别到基本情况时结束	继续条件不满足时结束
可转换性	每一个递归函数都可以被重写为迭代函数	-
特点	消耗内存和处理器性能，速度慢，但简洁优美(但是也会可能爆栈)	高效，但设计难度较大

[2] 链表结构

Linked Structure

我们学过什么

用数组实现集合：

- ArrayList：除在末尾外，添加/删除元素的时间复杂度为 $O(n)$
- Stack：时间复杂度为 $O(1)$
- ArraySet：添加/查找/删除元素的时间复杂度为 $O(n)$ （← 搜索成本）

HashSet 和 HashMap 的存取是无序的，也就是你用迭代器进行遍历拿出来的元素和插入的时候顺序不同。

但是如果存储有序，又只能用数组这种结构，但这种结构的缺点就是它的时间复杂度太大了。

因此我们可以尝试使用带链表的哈希，例如 LinkedHashSet 和 LinkedHashMap，他们俩提供 getFirst 和 getLast 等方法能够让我们操作最开始、最末尾的两个元素。用迭代器进行迭代的时候存取有序！

尝试制作一个链表结构

```
1 public static void main(String[] args) {
2     MyNode<String> node = new MyNode<>();
3     node.insert("Apple");
4     node.insert("Banana");
5     node.insert("Cat");
6     node.insert("Dog");
7     node.insert("Fish");
8     System.out.println(node);
9     node.deleteLast();
10    System.out.println(node);
11    System.out.println(node.getEach());
12 }
```

输出

```
1 MyNode{head=[Apple], tail=[Fish], size=5}
2 MyNode{head=[Apple], tail=[Dog], size=4}
3 [Apple,Banana,Cat,Dog]
```

需要有简单的增删查改....

```
1 public class MyNode<E> {
2     private Node head; // 储存头部
3     private Node tail; // 储存尾部
4     private int size;
5
6     public MyNode() {
7
8     }
9
10    // 增
11    public void insert(E e) {
12        size++;
13        if (isEmpty()) {
14            head = new Node(e, null, null);
15            tail = head;
16            return;
17        }
18        tail.next = new Node(e, null, tail);
19        tail = tail.next;
20    }
21
22    // 删
23    public E delete(E e) {
24        if (isEmpty() || exist(e)) return null;
25        Node temp = head;
26        while (temp.next != null) {
27            if (temp.data.equals(e)) break;
28            temp = temp.next;
```

```

29     }
30     if (temp.prev != null) temp.prev.next = temp.next;
31     if (temp.next != null) temp.next.prev = temp.prev;
32     size--;
33     return temp.data;
34 }
35
36 public E deleteLast() {
37     if (isEmpty()) return null;
38     size--;
39     if (tail == head) {
40         head = tail = null;
41         return head.data;
42     }
43     E data = tail.data;
44     tail = tail.prev;
45     tail.next = null;
46     return head.data;
47 }
48
49 // 查
50 public boolean exist(E e) {
51     if (isEmpty()) return false;
52     Node temp = head;
53     while (temp.next != null) {
54         if (temp.data.equals(e)) return true;
55         temp = temp.next;
56     }
57     return false;
58 }
59
60 // 改
61 public E set(E originData, E newData) {
62     if (isEmpty()) return null;
63     Node temp = head;
64     while (temp.next != null) {
65         if (temp.data.equals(originData)) break;
66         temp = temp.next;
67     }
68     E oldData = temp.data;
69     temp.data = newData;
70     return oldData;
71 }
72
73 // 是否为空
74 public boolean isEmpty() {
75     return head == null && tail == null;
76 }
77
78 @Override
79 public String toString() {
80     return "MyNode{" + "head=" + head + ", tail=" + tail + ", size=" + size + '}';
81 }

```



```

82
83     public int size() { // 大小
84         return size;
85     }
86
87     private class Node { // 自定义内部类
88         private E data; // 这个点的数据
89         private Node next; // 前一个点
90         private Node prev; // 后一个点
91
92         public Node(E data, Node next, Node prev) {
93             this.data = data;
94             this.next = next;
95             this.prev = prev;
96         }
97
98         @Override
99         public String toString() {
100             return "[" + data + "]";
101         }
102     }
103     public String getEach(){
104         StringBuilder res = new StringBuilder("[");
105         Node temp = head;
106         while (temp != null) {
107             res.append(temp.data);
108             if(temp.next != null)
109                 res.append(",");
110             temp = temp.next;
111         }
112         res.append("]");
113         return res.toString();
114     }
115 }

```

Q&A

[题目] When do you write test cases for “black box” testing? Before or after implementation?

翻译：你在什么时候为“黑盒”测试编写测试用例？是在实现之前还是之后？

- A. 在实现之前
- B. 在实现之后

答案：A

解释：黑盒测试是基于软件的需求规格说明书，不考虑程序内部结构和逻辑，主要关注软件的功能。在开始实现软件之前，需求规格说明书通常已经存在，基于此就可以编写黑盒测试用例，用于验证软件是否满足功能需求。所以应在实现之前编写黑盒测试用例。

[题目] Define references/pointers.

翻译：定义引用 / 指针。

答案：

- 指针是一个存储另一个变量内存地址的变量，通过它可以直接访问数据存储的内存位置。
- 引用是现有变量的别名，一旦初始化引用某个变量，就不能再引用其他变量

- 在程序设计中，假设我们有一个变量 `num`，可以这样定义：`int num = 10;`。如果我们再定义一个引用 `ref` 指向 `num`，即 `int &ref = num;`，那么 `ref` 就是 `num` 的别名。这意味着 `ref` 和 `num` 实际上是同一个数据在内存中的不同名字。
- 例如，对 `ref` 进行赋值操作，就如同对 `num` 赋值。如果执行 `ref = 20;`，那么 `num` 的值也会变为 20，因为它们指向同一块内存区域。

一旦初始化不能再引用其他变量

- 当引用 `ref` 被初始化为指向 `num` 后，就不能再让 `ref` 指向其他变量。比如，假设有另一个变量 `anotherNum` 定义为 `int anotherNum = 30;`，我们不能像对指针那样重新赋值让 `ref` 指向 `anotherNum`。如果尝试 `ref = anotherNum;`，这并不是让 `ref` 指向 `anotherNum`，而是将 `anotherNum` 的值赋给了 `ref` 所引用的变量（也就是 `num`），此时 `num` 的值会变为 30。
- 与之对比，指针是可以重新赋值指向不同变量的。例如，对于指针 `int *ptr = #`，之后可以执行 `ptr = &anotherNum;`，让 `ptr` 从指向 `num` 变为指向 `anotherNum`。但引用不具备这种重新指向其他变量的能力，它从初始化那一刻起，就与被引用的变量紧密绑定。

[题目] 解释垃圾回收

在 Java 中，如果一个对象没有被任何外部引用指向它（即它处于不可达状态），即使该对象内部引用了其他对象，这个对象及其内部所引用的所有对象通常都会被垃圾回收机制回收。

■ [Week 7 Lecture 15,16]

讲了如何用 front (head) 和 back 来用链表实现 Stack 和 Queue

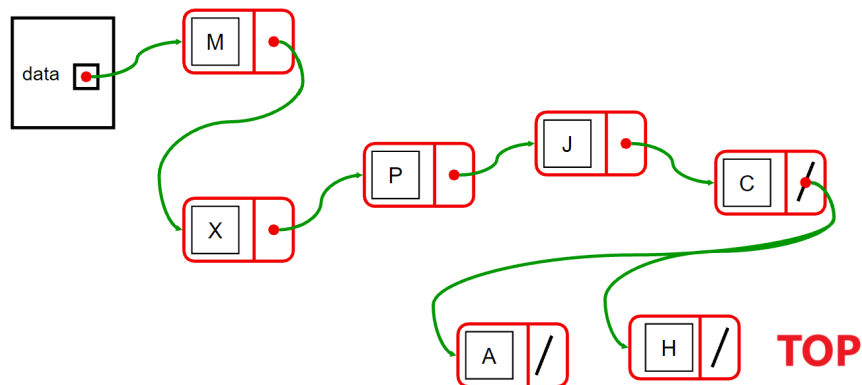
使用带头链表实现的 **栈 Stack**

使用带头链表实现的 **队列 Queue**

[1] 栈 LIFO

Last in first out LIFO

如果我们用链表实现栈，那么TOP就是后进的元素，那么这样做有什么坏处呢？（当我们试图插入的时候，就需要 $O(n)$ 时间了！）

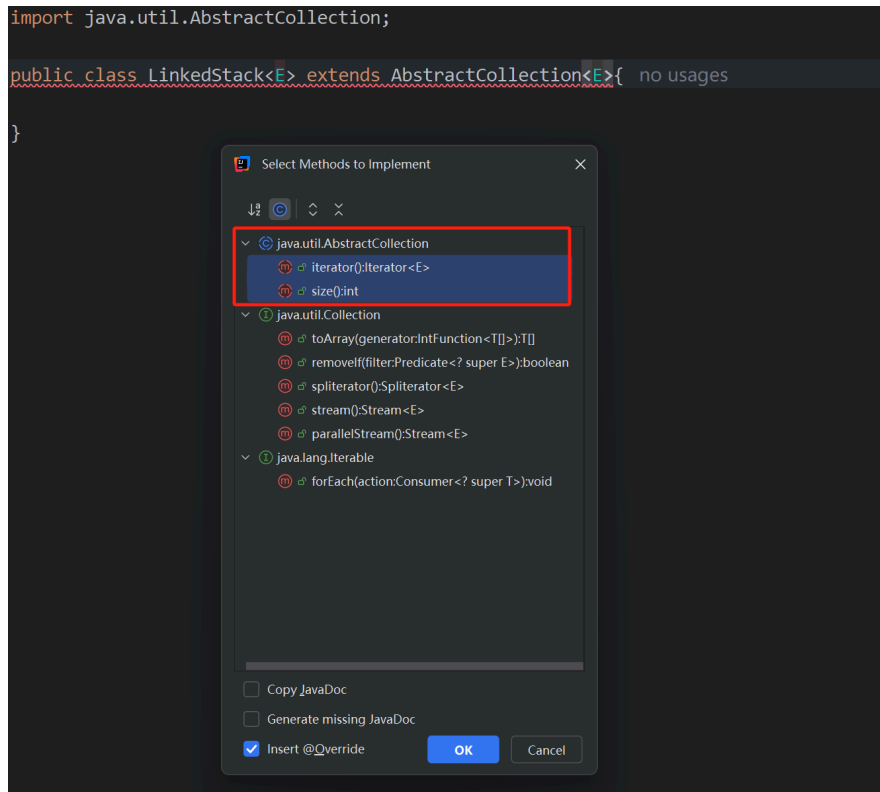


因此，我们应该把TOP放在头部，也就是 data 部分

[2] 实现 LinkedStack

根据PPT内容改写

如果我们需要实现 `LinkedStack`，可以继承抽象集合类，然后复写所有抽象方法...



```
1  import java.util.AbstractCollection;
2  import java.util.Iterator;
3
4  public class LinkedStack<E> extends AbstractCollection<E>{
5      private Node<E> data; // head
6      private static class Node<E>{
7          Node<E> next;
8          E element;
9      }
10     private int size;
11     public boolean isEmpty(){
12         return size == 0;
13     }
14     public E peek(){
15         return data.element;
16     }
17     public E pop(){
18         Node<E> temp = data;
19         data = data.next;
20         size--;
21         return temp.element;
22     }
23     public void push(E e){
24         if(size == 0)
25         {
26             data = new Node<E>();
27             data.element = e;
28             size++;
29             return;
30         }
31         Node<E> tempNode = data;
```

```

32     data = new Node<E>();
33     data.element = e;
34     data.next = tempNode;
35     size++;
36 }
37 @Override
38 public Iterator<E> iterator() {
39     return new Iterator<E>() {
40         int currentIndex = 0;
41         Node<E> temp = data;
42         @Override
43         public boolean hasNext() {
44             return currentIndex++<size;
45         }
46         @Override
47         public E next() {
48             Node<E> retNode = temp;
49             temp = temp.next;
50             return retNode.element;
51         }
52     };
53 }
54 @Override
55 public int size() {
56     return size;
57 }
58 }

```

测试类输出

```

1 public static void main(String[] args) {
2     LinkedList<Integer> s = new LinkedList<Integer>();
3     s.push(1);
4     s.push(2);
5     s.push(3);
6     for (Integer integer : s) {
7         System.out.println(integer);
8     }
9 }
10

```

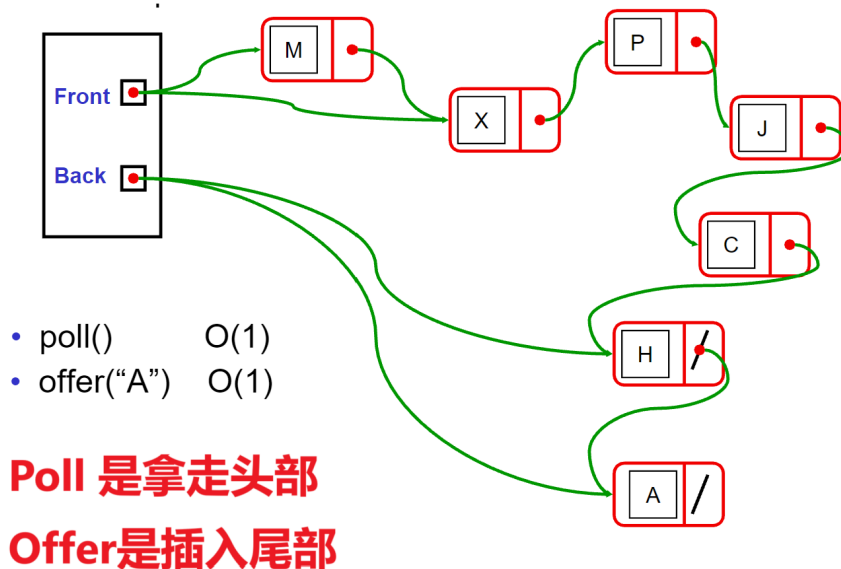
```

1 3
2 2
3 1

```

[3] 实现 LinkedList

按照之前的逻辑很容易也实现LinkedList，但是如果优化的话，就可以定义back和front两个变量，这样时间复杂度会降到 $O(1)$ 。



这次 Queue 接口，该接口同样 extends `AbstractCollection<E>` 继承了抽象集合类，因此我们需要重写两者...

```

5  public class LinkedList<E> extends AbstractQueue<E>{ no usages
6
7

```

Select Methods to Implement

- java.util.AbstractCollection
 - iterator():Iterator<E>
 - size():int
- java.util.Queue
 - offer(e:E):boolean
 - poll():E
 - peek():E
- java.util.Collection
 - toArray(generator:IntFunction<T[]>):T[]
 - removeIf(filter:Predicate<? super E>):boolean
 - spliterator():Spliterator<E>
 - stream():Stream<E>
 - parallelStream():Stream<E>
- java.lang.Iterable
 - forEach(action:Consumer<? super T>):void

☐ Copy _JavaDoc
☐ Generate missing JavaDoc
☒ Insert @Override

OK Cancel

```

1  import java.util.AbstractQueue;
2  import java.util.Iterator;
3
4  public class LinkedList<E> extends AbstractQueue<E>{
5      private int size = 0;

```

```

6     private Node<E> front;
7     private Node<E> back;
8     private static class Node<E>{
9         Node<E> next;
10        Node<E> prev;
11        E element;
12    }
13    @Override
14    public Iterator<E> iterator() {
15        return new Iterator<E>() {
16            int counter = 0;
17            Node<E> current = front;
18            @Override
19            public boolean hasNext() {
20                return counter++ < size;
21            }
22
23            @Override
24            public E next() {
25                Node<E> temp = current;
26                current = current.next;
27                return temp.element;
28            }
29        };
30    }
31
32    @Override
33    public int size() {
34        return size;
35    }
36
37    @Override
38    // front| a->b->c->d-> ... |back [next]
39    public boolean offer(E e) {
40        if(size==0){
41            Node<E> newNode = new Node<>();
42            newNode.element=e;
43            front = newNode;
44            back = newNode;
45            size++;
46            return true;
47        }
48        Node<E> tempNode = back;
49        back = new Node<>();
50        back.element=e;
51        tempNode.next = back;
52        back.prev = tempNode;
53        size++;
54        return true;
55    }
56
57    @Override
58    public E poll() {

```

```

59     Node<E> tempNode = front;
60     front = tempNode.next;
61     if(front != null)
62         front.prev = null;
63     size--;
64     return tempNode.element;
65 }
66
67 @Override
68 public E peek() {
69     return front.element;
70 }
71 }

```

[4] ArraySet & SortedArraySet

操作方法	时间复杂度
<code>get</code>	$O(1)$
<code>set</code>	$O(1)$
<code>remove</code>	$O(n)$
<code>add (at i)</code> (需移动元素, 可能要扩容)	$O(n)$ (最坏和平均情况)
<code>add (at end)</code> (多数情况)	$O(1)$
<code>add (at end)</code> (扩容时, 最坏情况)	$O(n)$
<code>add (at end)</code> (均摊平均, 每次扩容翻倍时)	$O(1)$

ArraySet 操作成本

- **ArraySet 特性:** 元素顺序不重要, 不允许重复元素。
- **操作算法及成本:**
 - `contains(value)` 通过遍历数组判断元素是否存在
 - `add(value)` 在数组末尾添加元素 (必要时翻倍扩容), 添加前需检查元素是否已存在
 - `remove(value)` 找到元素后用数组末尾元素替换并更新大小。
 - 这三个操作时间复杂度均为 $O(n)$ 。

但是这三个操作时间复杂度均为 $O(n)$ 还是太大了, 因此, 可以考虑**二分查找** (如果ArraySet有序)

二分查找

- **原理:** 适用于有序数组。每次查找时, 先比较目标元素与数组中间元素, 若相等则返回; 若目标元素小于中间元素, 则在左半部分继续查找; 若大于中间元素, 则在右半部分查找。
- **算法实现:** 通过 while 循环, 不断调整查找范围 (low 和 high), 直至找到目标元素或确定目标元素不存在。
- **时间复杂度:** 用 $T(n)$ 表示二分查找在 n 个元素集合中的时间复杂度, 其递归公式为

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ T(n/2) + 1, & \text{otherwise} \end{cases}$$

通过代换法可证明其时间复杂度为 $O(\log n)$ 。

```
private boolean contains(Object item){
    Comparable<E> value = (Comparable<E>) item;
    int low = 0; // min possible index of item
    int high = count-1; // max possible index of item
                    // item in [low .. high] (if present)

    while (low <= high){
        int mid = (low + high) / 2;
        int comp = value.compareTo(data[mid]);
        if (comp == 0) // item is present
            return true;
        if (comp < 0) // item in [low .. mid-1]
            high = mid - 1; // item in [low .. high]
        else // item in [mid+1 .. high]
            low = mid + 1; // item in [low .. high]
    }
    return false; // item in [low .. high] and low > high,
                  // therefore item not present
}
                                     low      mid      high
```

这里的 `Comparable` 实际上是调用对应类的（复写）`compareTo` 方法（如果没有就调用父类 `Object` 的 `compareTo` 方法）\

```
1 @Override
2 public int compareTo(Student o) {
3     return this.age - o.age;
4 }
```

假设 `Student` 类中复写了这个方法，那么就可以比较了，这里是通过年龄比较

上述方法中变量 `value` 就是 `Comparable` 下的泛型 `E` 类...

使用二分查找的 `SortedArraySet` 的成本

- **SortedArraySet 优势：** `contains` 操作时间复杂度为 $O(\log n)$ ，相比 `ArraySet` 的 $O(n)$ 更快。
- **局限性与优化：** 频繁调用 `add` 和 `remove` 操作时，`SortedArraySet` 与 `ArraySet` 效率相近，因为维护数组有序会增加成本；若仅构建一次集合，后续频繁调用 `contains`，则 `SortedArraySet` 更具优势。可通过一次性排序构建集合，如 `SortedArraySet(Collection<E> col)` 构造函数，先将集合元素放入数组，再使用 `Arrays.sort(data)` 进行排序。

[5] 提升 `SortedArraySet` 的速度

如果你需要频繁调用 `add()` 和/或 `remove()` 方法来处理大量元素，那么有序数组集（`SortedArraySet`）并不比普通数组集（`ArraySet`）好！

- 二者时间复杂度均为 $O(n)$ 。
- 要么在查找上付出代价。
- 要么在维持元素有序上付出代价。

如果你只需构建一次集合，然后频繁调用 `contains()` 方法，那么有序数组集（`SortedArraySet`）比普通数组集（`ArraySet`）要好得多。

- 有序数组集的 `contains()` 方法时间复杂度为 $O(\log(n))$ 。

但是，如何快速构建这个集合呢？

- 逐个添加元素。

■ [Week 8 Lecture 17,18,19]

排序算法

其中，课件上标注**选择排序**、**插入排序**、**冒泡排序**是慢排序，时间复杂度均为 $O(n^2)$

归并排序 (Merge Sort)、**快速排序**是快排序

其中归并排序运用了**分治(Divide and Conquer)**思想。因此他们都可以认为是分治排序。

排序算法	最好时间复杂度	最坏时间复杂度	平均时间复杂度
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$

选择排序、插入排序、冒泡排序、分治排序在INT102的课程中都讲过了，那么这里就说说快速排序。

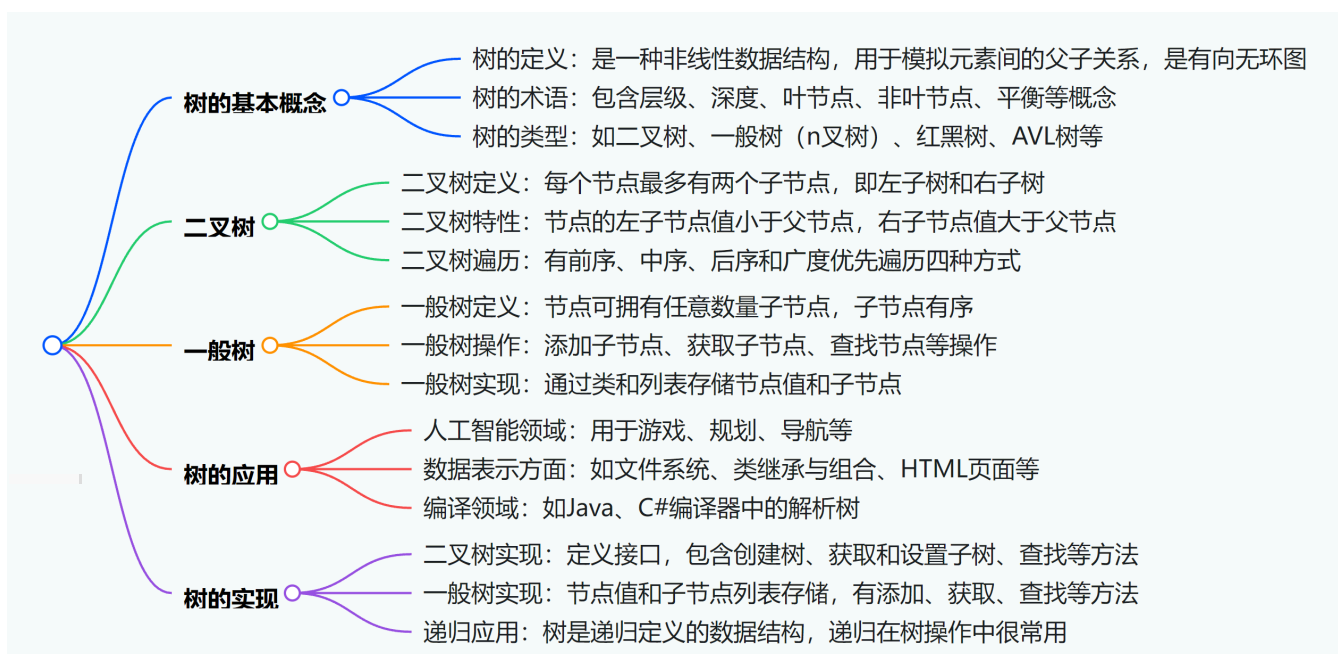
[1] 快速排序

快速排序实际也是一种分治。

```
1  import java.util.Arrays;
2  import java.util.Collections;
3  import java.util.Random;
4
5  public class QuickSort {
6      private static final Random rand = new Random(System.currentTimeMillis());
7      public static void sort(int[] arr) {
8          if(arr == null || arr.length < 2) {
9              return;
10         }
11         sort(arr,0,arr.length-1);
12     }
13     private static void sort(int[] arr, int left, int right) {
14         if(left >= right) return;
15         int pivot = getPivot(arr,left,right);
16         sort(arr,left,pivot-1);
17         sort(arr,pivot+1,right);
18     }
19     private static int getPivot(int[] arr, int left, int right) {
20         int i = left;
21         int j = right;
22         // 随机一个作为pivot，并移动到左边
23         swap(arr,left,rand.nextInt(right - left + 1) + left);
24         int pivot = arr[left];
25         while(i<j){
```

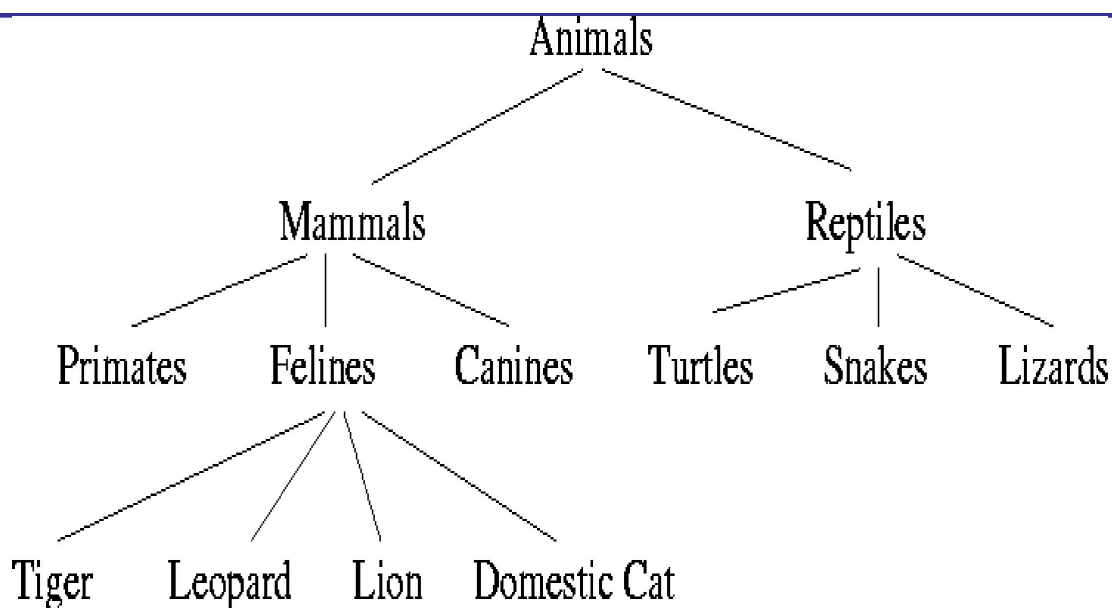
```
26         while(i<j && arr[j] > pivot){j--;}
27         while(i<j && arr[i] <= pivot){i++;}
28         swap(arr,i,j);
29     }
30     // i == j now
31     swap(arr,i,left);
32     return i;
33 }
34 private static void swap(int[] arr, int i, int j) {
35     int temp = arr[i];
36     arr[i] = arr[j];
37     arr[j] = temp;
38 }
39 public static void main(String[] args) {
40     int[] arr = {5,1,1,2,0,0};
41     Arrays.parallelSort(arr);
42     QuickSort.sort(arr);
43     for (int num : arr) {
44         System.out.print(num + " ");
45     }
46 }
47 }
48 }
```

■ [Week 9 Lecture 20, 21, 22]

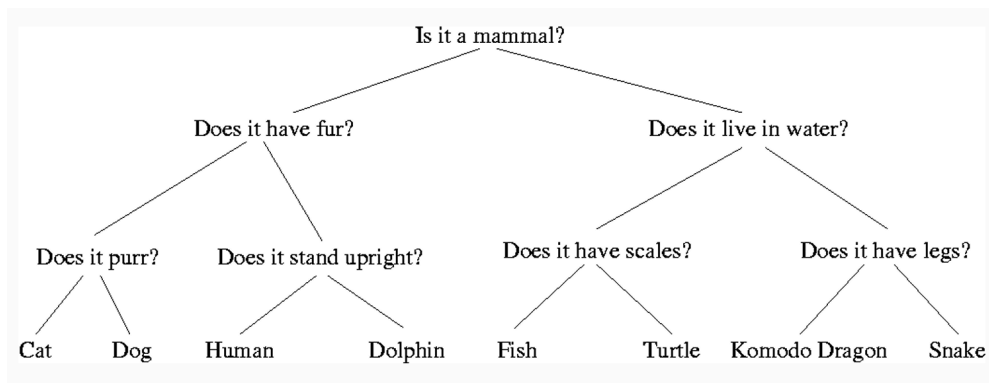


[1] 介绍树

树有很多种...例如分类树 (Taxonomy Tree)

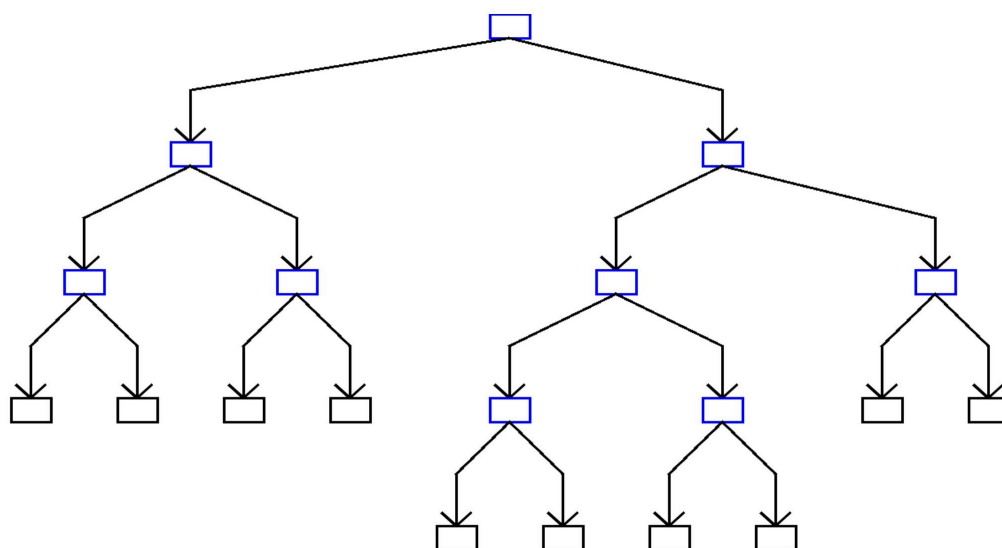


再比如, 决策树 (机器学习中的)

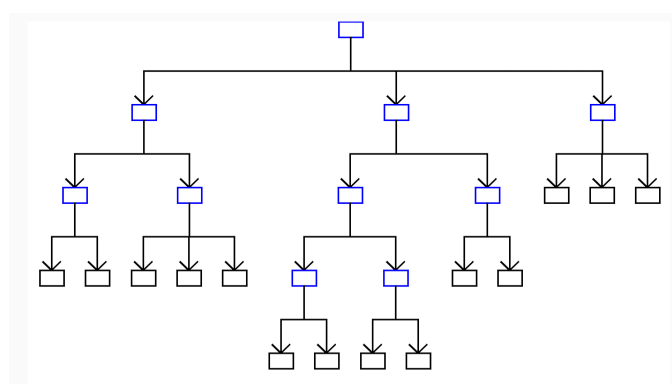


树是**非线性数据结构 (Non-linear data structure)**

最常见的树就是二叉树 (Binary Tree)，简而言之只有两个分支（对于每个节点而言），可以叫做左子节点、右子节点。



当然，如果他们没有左子节点、右子节点，就可以设对应的子节点为 `null`。



这张图就不是二叉树了

树究竟是什么？

从数学角度来看：是一种“有向无环图”(DAG)。任意两个节点之间最多存在一条路径

[2] 术语

层级 (Level)：相当于一个值所在的“行”。

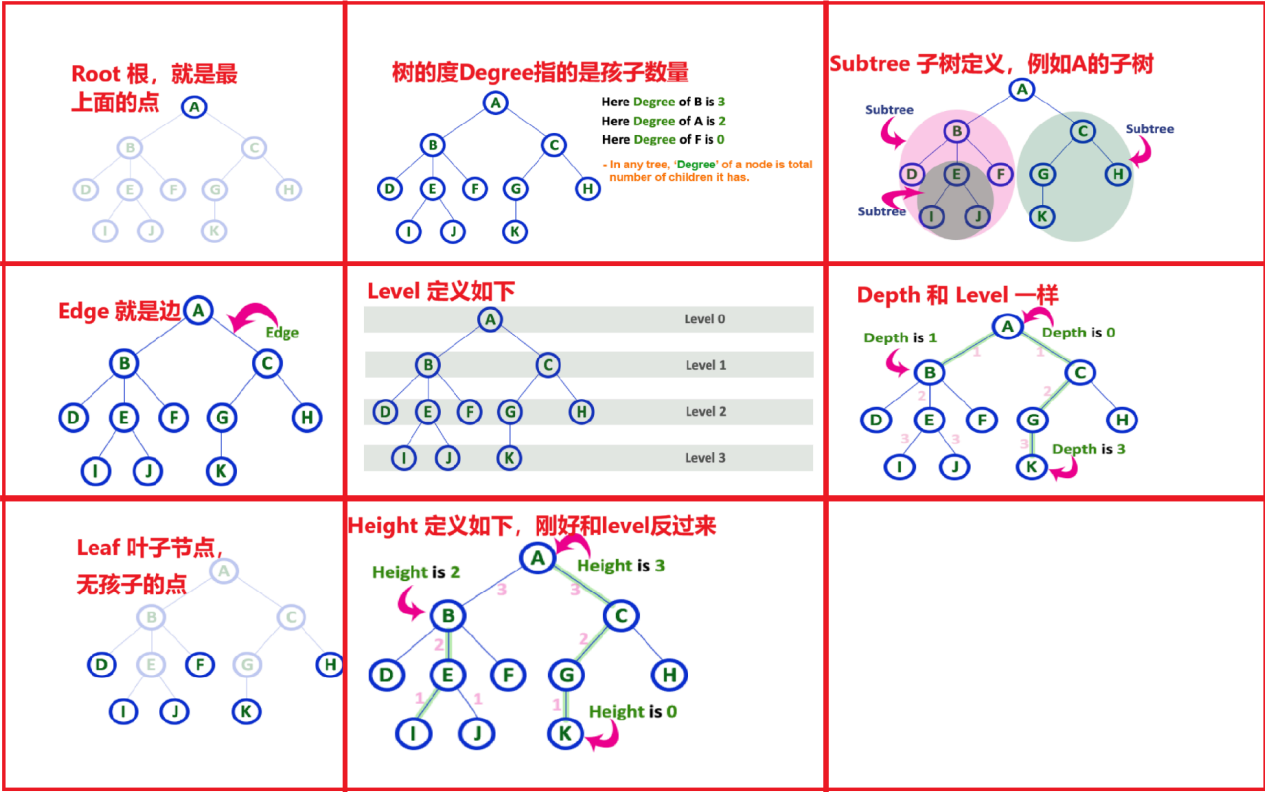
深度 (Depth)：树中层级的数量。

叶节点 (Leaf Nodes): 没有子节点的节点。

非叶节点 (Non - leaf Nodes)

平衡 (Balanced): 对于某个n值，所有叶节点都位于第n层和第(n + 1)层（且在底层“靠左”聚集）。

Picture from INT102 Notes



树有多少种类型？

红黑树、AVL树...

不同类型的树有不同用途：提升速度、提高内存利用率、适配特定问题

但我们将研究以下三种：二叉树、一般树（n叉树）、AVL树

树有什么作用？

人工智能领域：用于规划、导航、游戏等方面。

事物表示方面：简单文件系统、类继承与组合、分类、HTML页面、语言分析树、在Java、C# 等编译器中是关键部分、3D图形（如二叉空间分割树 BSP）。

[3] Q&A

[题目 1] Tree represents an efficient 1 - dimensional data structure. (T or F?)

翻译：树表示一种高效的一维数据结构。（对或错？）

A. T（对）

B. F（错）

答案：B

解释：树是一种非线性的数据结构，它用于表示层次关系，并非一维数据结构。一维数据结构如数组、链表等，数据元素呈现线性排列。所以该说法错误，答案为 B。

[题目 2] **A leaf node in a tree has no children. (T or F?)**

翻译：树中的叶节点没有子节点。（对或错？）

A. T（对）

B. F（错）

答案：A

解释：在树的结构定义中，叶节点指的就是那些没有子节点的节点，它处于树的最底层分支末端。所以该表述是正确的，答案为 A。

[题目 3] **Binary tree has no ordering upon its sibling nodes. (T or F?)**

翻译：二叉树的兄弟节点之间没有顺序。（对或错？）

A. T（对）

B. F（错）

答案：F

解释：在二叉树中，每个节点最多有两个子节点，分别被定义为左子节点和右子节点，这就表明了二叉树的兄弟节点（同一父节点下的子节点）是有顺序之分的，左子节点在顺序上先于右子节点。所以该说法错误，答案是 B。

[题目 4] **Name 3 applications for tree.**

翻译：列举树的三个应用场景。

1. **File System Directories**（文件系统目录）：文件系统采用树状结构来组织文件和文件夹。根目录类似树的根节点，文件夹可以看作是内部节点，文件则如同叶节点。这种结构便于对文件进行分类、管理和检索。例如，在计算机的硬盘中，不同类型的文件被放置在不同的文件夹下，通过树状的目录结构，用户能够快速找到所需文件。
2. **Expression Evaluation**（表达式求值）：对于算术表达式等，可以用二叉树来表示。操作数作为叶节点，运算符作为内部节点。通过对这种表达式树的遍历（如中序遍历得到中缀表达式，前序遍历得到前缀表达式，后序遍历得到后缀表达式），可以实现表达式的求值。例如，对于表达式 $(3 + 5) * 2$ ，可以构建相应的表达式树进行求值。
3. **Searching and Sorting**（搜索和排序）：像二叉搜索树这种特殊的树结构，在搜索和排序方面有广泛应用。二叉搜索树的特点是左子树所有节点的值小于根节点的值，右子树所有节点的值大于根节点的值。利用这个特性，在二叉搜索树中进行查找、插入和删除操作的平均时间复杂度为 $O(\log n)$ ，可以高效地实现搜索和排序功能。例如，在数据库的索引中常使用类似二叉搜索树的结构来加速数据的查找。

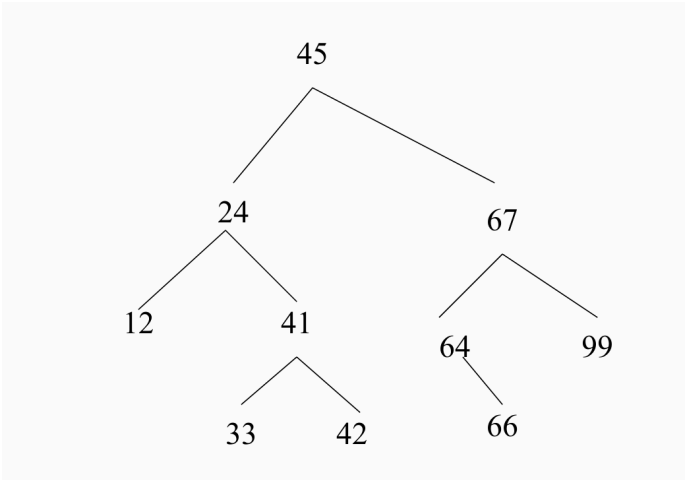
[题目 5] **Relationship among recursive calls can be expressed in what type of tree?**

翻译：递归调用之间的关系可以用哪种类型的树来表示？

答案：Call Tree（调用树）

解释：在程序执行递归调用时，每一次递归调用就像是树中的一个节点，调用关系形成了层次结构，类似于树的父子关系。初始的调用是根节点，后续的递归调用作为子节点，清晰地展示了递归调用的过程和层次，这种用于表示递归调用关系的树就被称为调用树。所以递归调用之间的关系可以用调用树来表达。

[4] BST 二叉搜索树



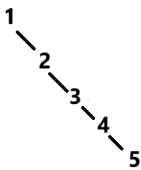
二叉搜索树定义

- 二叉搜索树是一种二叉树，其中每个节点都有一个键。
- 一个节点的左子节点（如果存在）的键小于其父节点的键。
- 一个节点的右子节点（如果存在）的键大于其父节点的键。
- 根节点的左子树和右子树也都是二叉搜索树。
- 平均情况下，插入、查找和删除操作的时间复杂度为 $O(\log N)$ 。

最坏情况

- 操作的时间复杂度可能退化为 $O(N)$ ——这是最坏情况！

如图是退化树（Degenerated Tree）



- 会退化为链表：
 - 当按键值升序插入时，所有键值都在右侧。
 - 当按键值降序插入时，所有键值都在左侧。
- 理想情况是先插入中间值，然后依次插入后续中间值等。
- 随机顺序的插入效果也相当不错。

遍历方式

INT104 有提到二叉树，或者BST的三种遍历方式

遍历：Traversal

递归：Recursion（函数调用自身）

迭代：Iteration（while 循环等）

1. 前序遍历 Preorder，输出顺序：中左右

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return; // 结束条件
3     tempList.add(nodes.data); // 输出
4     if (nodes.left != null) getTree(nodes.left); // 然后访问左节点
5     if (nodes.right != null) getTree(nodes.right); // 然后访问右节点
6 }
```

2. 中序遍历 Inorder，输出顺序：左中右。如果是平衡二叉树，那么输出就是从小到大，这是最常见的遍历方式

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return;
3     if (nodes.left != null) getTree(nodes.left);
4     tempList.add(nodes.data); // 输出
5     if (nodes.right != null) getTree(nodes.right);
6 }
```

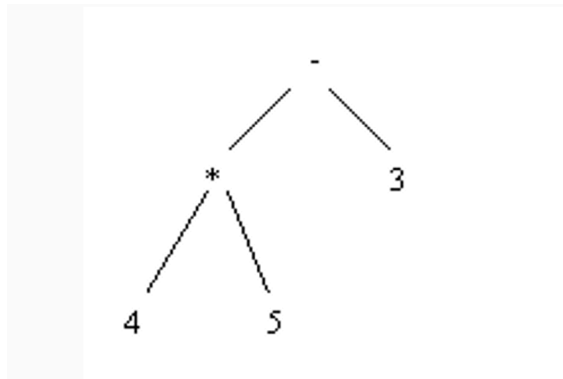
3. 后序遍历 Postorder，输出顺序：左右中

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return;
3     if (nodes.left != null) getTree(nodes.left);
4     if (nodes.right != null) getTree(nodes.right);
5     tempList.add(nodes.data); // 输出
6 }
```

[5] 表达式树

给定一个中序表达式，如何转为二叉树？

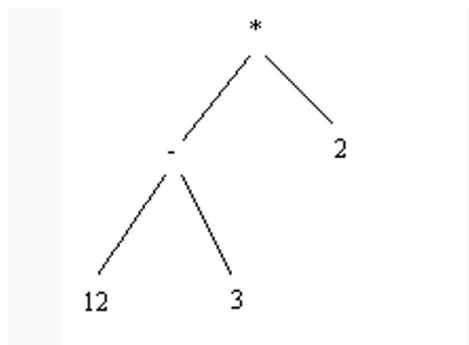
手写出下面树的 Inorder 遍历方式（这棵树代表 $(4*5)-3$ 也就是 $4*5-3$ ）



Inorder，是中序遍历，结构是左（递归）中（输出）右（递归）

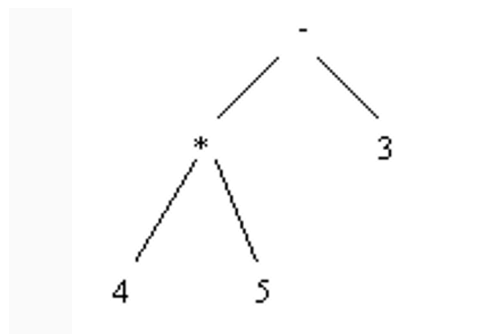
因此输出：4 * 5 - 3

同理，手写下面树的 Inorder 遍历方式（这棵树代表 $(12-3)*2$ ）



输出：12 - 3 * 2

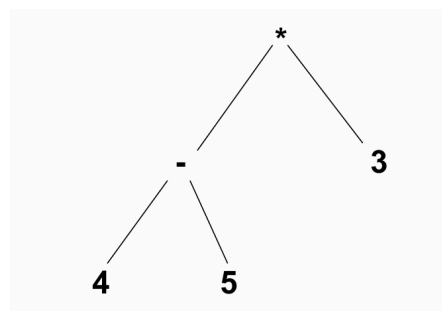
同理，手写下面树的 preorder（这棵树代表 $4*5-3$ ）



preorder 是 前序遍历，中（输出）左（递归）右（递归）

输出：- * 4 5 3

给定 $(4-5)*3$ 你如何构建出正确的前序遍历树？



求中序遍历？答案：`4 - 5 * 3`；因此中序表达式：`(4-5)*3`

求前序遍历？答案：`*-453`；

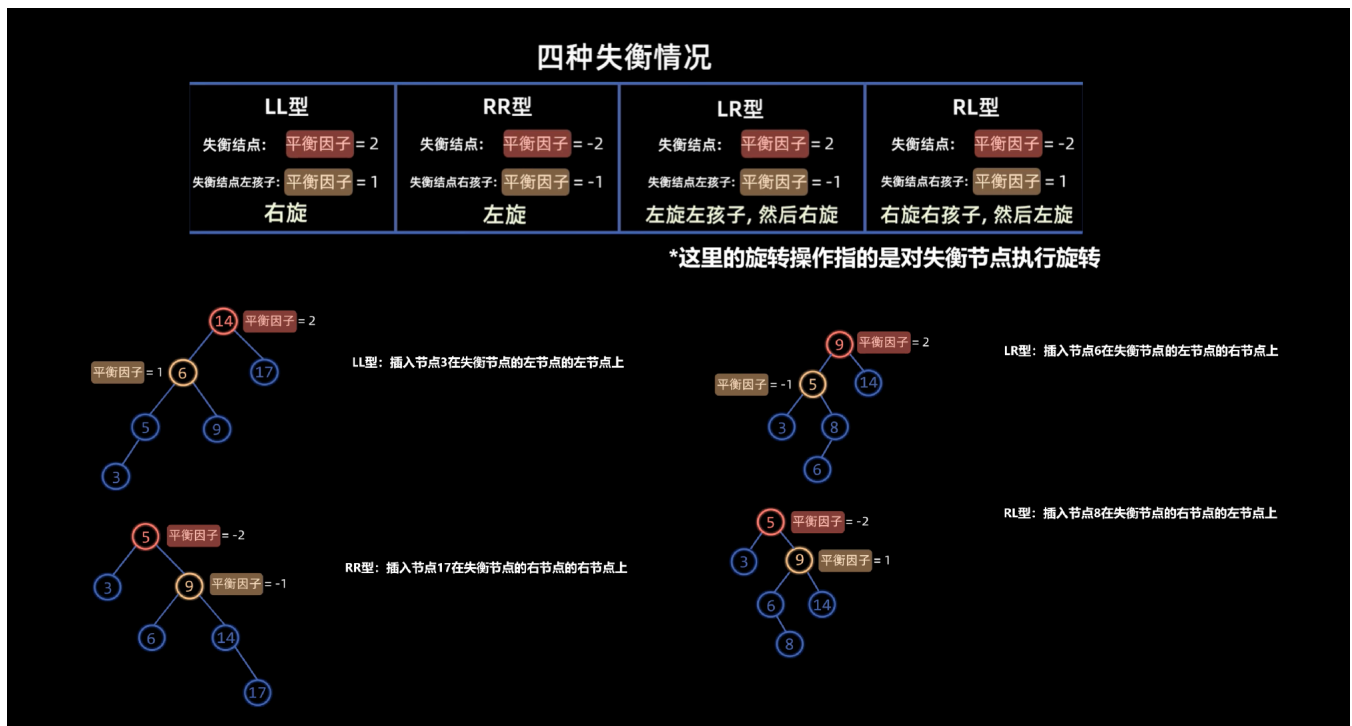
求后序遍历？答案：`45-3*`，这个也叫做逆波兰式

[6] 平衡二叉树之 AVL 树

为了防止二叉树BST退化为链表，插入的时候需要适当做**旋转（rotation）**操作

可以观看[这个视频了解AVL树以及其旋转操作。](#)

```
1  /*
2  * 1.    点A的平衡因子定义如下：A的左子树高度 - A的右子树高度
3  *      如果子树为null，那么计算而言，高度视为0
4  * 2.    失衡节点因子=2，其左节点因子=1；对失衡节点进行左旋      LL
5  *      失衡节点因子=2，左节点因子=-1；左节点左旋后，节点右旋      LR
6  *      失衡节点因子=-2，其右节点因子=-1；对失衡节点进行左旋      RR
7  *      失衡节点因子=-2，其右节点因子=1；右节点右旋，节点左旋      RL
8  * */
```



对于每一个节点 Node n，可以定义它的高度 `height = 1`，初始是1意味着我们现在把他看成一个树。

插入操作的时候，找到待插入点，然后插入。如果对于父节点F而言，n是插入左边，并且右边是空的，也就是父节点F需要更新高度。

高度更新为2（也就是n的高度1然后+1）。如果右边不是空的，那么父节点的右边显然高度 $\geq$ n的高度，这时候不需要更新高度。

根据高度可以判断出父节点的平衡因子，来决定父节点是否需要旋转。

具体代码查看INT104文件夹下的 `Algorithm.pdf`

复杂度分析

Algorithm	Average	Worst case
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$
Space	$O(n)$	$O(n)$

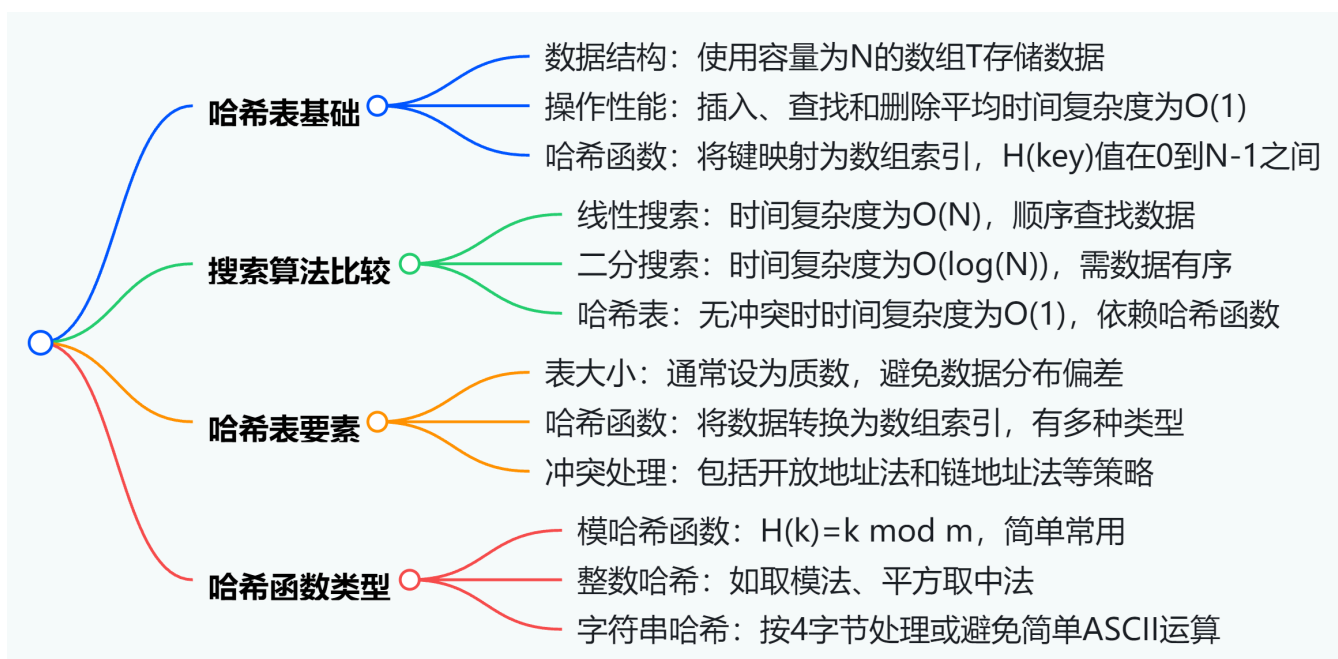
[7] 一般树

一般树就是可以有任意子节点数量的树。看看PPT是如何实现的。

```
public class GeneralTree<V> { 定义了一个树，并且指定泛型V
    包含一个值 private V value;
    和对应的子节点 private List< GeneralTree<V> > children;

    public GeneralTree(V value) { 节点结构就是 值+一个ArrayList
        this.value = value;        ArrayList里面存放的是子节点或者说是子树（可以查找root值）
        this.children = new ArrayList< GeneralTree<V> >();
    }
    ...
}
```

■ [Week 10 Lecture 23]



[1] 什么是哈希表?

哈希表也就是散列表。不发生哈希冲突的情况下：插入、查找、删除时间复杂度均为 $O(1)$

查看这个视频初步了解[哈希表](#)。然后进一步了解[哈希表](#)。

什么是除留余数法?

其实就是取模。

如何获得模?

一般而言，直接是表长 `hashTable.size()`，这样返回的范围就是 `[0,hashTable.size()-1]`

取什么样的模数能够降低冲突?

经过数学证明，取模的数不仅仅可以是表长 `hashTable.size()`，如果换成 $\leq \text{hashTable.size()}$ 的最大质数会更好！

比如表长为 8，那么取模数为 7 会更好！

什么是装填因子?

也就是空间利用率。

如果输入数据不是整数怎么办?

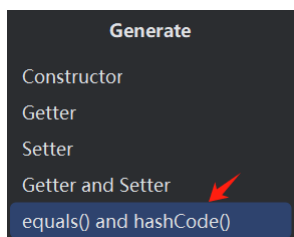
如果是String，可以相加每一位的 `ASCII` 码值然后再取模。（糟糕的方法）

在Java中 `String` 有特定的哈希方法。

如果是自定义类，必须重写 `equals` 和 `hashCode` 方法才能正常使用标准库的某些哈希集合类例如 `HashSet` 和 `HashMap`。

`equals` 方法用于判断哈希冲突时，两个数据是否相等。而 `hashCode` 方法就是获取当前对象哈希值。如果没有重写就默认使用 `Object` 类中的 `equals` 和 `hashCode` 方法。

而且 Java 底层解决哈希冲突的方式是红黑树和链表。



字符串哈希

PPT介绍：

字母和数字的八进制范围在 0101 到 0172 之间

所以关键信息都在最低 6 位中

OCT	101	OCT	172
BIN	0100 0001	BIN	0111 1010

1. 按每次 4 字节处理输入字符串，把每个 4 字节块转成长整型值。()
2. 对这些长整型值求和。
3. 用取模运算将和映射到 0 至 `.size()-1` 范围。也可按需调整每次处理字节数。

其他整数哈希方法

PPT介绍了一种平方取中法（Mid-square Method）。

[2] 如何解决哈希冲突？

线性探测法

PPT方法

线性探测法（linear probing），

当遇到冲突，逐个从哈希表中**往后**寻找空位置插入。如果到表尾就重新到表头。当然，插入如此，查找也如此。对于删除操作，就需要额外标记。但是会遇到**堆积问题**

属于开放地址法

二次探测法

（PPT方法）

二次探测（平方探测法，quadratic probing）

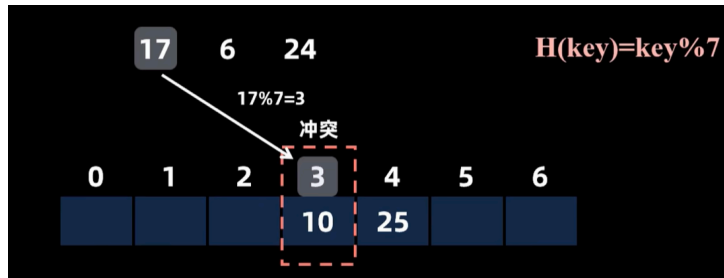
为了**缓解**线性探测法的问题，引入了二次探测法。

$\text{hash}(k, i) = (\text{hash}(k, i) + c1 * i + c2 * i * i) \bmod N$ 一般而言 $c1 = 0$ 并且 $c2 = 1$

冲突时按照 $+1^2, +2^2, +3^2 \dots$ 的顺序来探测

因为不一定能探测到所有距离，但是有研究证明如果表长是某个 $4K + 3$ 的质数那么就一定可以探测到所有位置，因此如果使用这种方法，表长一半是使用这类的数，例如 7。然后模数使用表长。

例子：



哈希表大小 $N = 7$ ，先定义哈希函数 $\text{hash1}(k)$ ，这里简单采用 $k \% N$ ，并取 $c1 = 0$ ， $c2 = 1$ 。

- 计算初始哈希地址

根据 $\text{hash1}(k)$ ，对于 $k = 17$ ， $\text{hash1}(17) = 17 \% 7 = 3$ 。但位置 3 已被 10 占用，发生冲突，开始二次探测。

- 第一次探测 ($i = 1$)

根据二次探测公式 $\text{hash}(k, i) = (\text{hash1}(k) + c1 * i + c2 * i * i) \% N$ ，此时 $c1 = 0$ ， $c2 = 1$ ， $i = 1$ ，则 $\text{hash}(17, 1) = (3 + 0 * 1 + 1 * 1 * 1) \% 7 = 4$ 。位置 4 被 25 占用，继续探测。

- 第二次探测 ($i = 2$)

$\text{hash}(17, 2) = (3 + 0 * 2 + 1 * 2 * 2) \% 7 = 7 \% 7 = 0$ 。位置 0 为空，所以将 17 插入到位置 0。

- 最终数组变为 [17, NAN, NAN, 10, 25, NAN, NAN]。

会导致次聚集现象。属于开放地址法。

双散列

PPT方法

双散列（双重哈希，Double Hashing）， $\text{hash}(k, i) = (\text{hash1}(k) + i * \text{hash2}(k)) \% N$ ，可避免聚集问题
例如长度为 $N = 11$ 的数组插入数据 14，那么 $\text{hash1}(14) = 3$ 无冲突直接插入。

再插入数据 36， $\text{hash1}(36) = 3$ 发现冲突！这时候要计算 $\text{hash2}(36)$ ，那么 hash2 怎么计算呢？

$$\text{hash2}(k) = 1 + k \% (N - 1)$$

因此 $\text{hash2}(36) = 1 + 36 \% 10 = 7$ 。

就可以进行探测了！第一次探测 $i = 1$

$$\text{hash}(k, i) = \text{hash}(36, 1) = (\text{hash1}(36) + 1 * \text{hash2}(36)) \% 11 = 3 + 7 \% 11 = 10$$

因此放入位置 10

如果位置 10 也有冲突，就进行第二次探测也就是 $i = 2$ ，继续套公式，继续使用 hash1 和 hash2

属于开放地址法

拉链法

最简单，其实就是变成一个链表。然后冲突发生后挨个查找。

平方取中法

(PPT方法)

平方取中法 (Mid - square Method)，也就是：对于一个属 87431 要存到大小为 1000 的哈希表中，可以让他自己平方，然后取中间的三位数。

EXAMPLE OF MID SQUARE METHOD

M= 1000 (0-999)

x= 87431

Hash Function=> $f_D(x) = (x^2)$

$f_D(87431) = (87431)^2 = 7644179761$

= 417 or 179 can be the answer where key or x will be stored

INDEX	KEYS
0	
1	
2	
.	
417	87431
.	
179	

[3] 实现简单哈希表

查看 INT102文件夹下的 Algorithm.pdf

■ [Week 11 Lecture 24, 25, 26 27]

图论大部分在INT102的课程中涉及了，部分内容已省略

基本介绍图

一些基本英语单词：

- 关联 Incident
- 邻接 Adjacency
- 分量 Component

简单图和非简单图

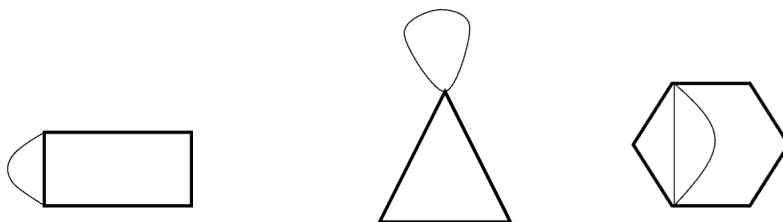
Simple graph 简单图

Non-simple graph 非简单图

简而言之，简单图就是 —— 对于任意顶点对 (x, y) ，只存在至多一条边链接他们俩。并且不存在自环。

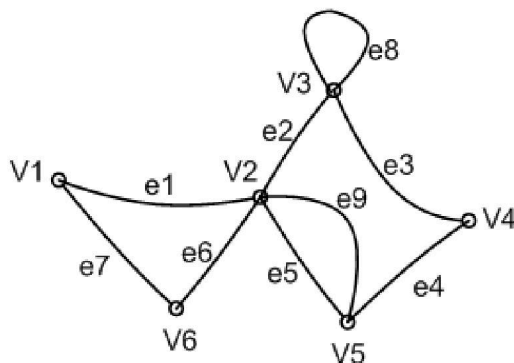


非简单图的例子如下：



例如如下的例子中， $(V2, V5)$ 存在两条边 $e5$ 和 $e9$ 连接他们俩。因此此图不是简单图。

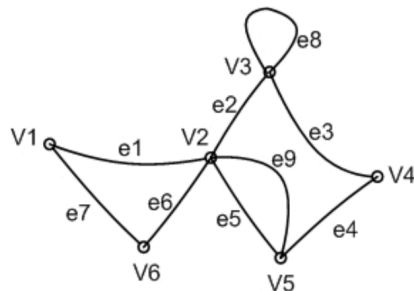
$e5$ 和 $e9$ 被认为是multiple edge



加权图 weighted graph

加权图就是带有边的图。

图的度



$$d(V1) = 2 \text{ and } d(V3) = 4$$

其实就是这个点有多少个连接的边（仔细看V3）

无向图的**度等于边数的两倍**，同理有向图也是（不过是入度+出度=2 |A|）。

有向图中的度分为入度和出度：

有向图的入度（in-degree）表示为 $d^-(V)$

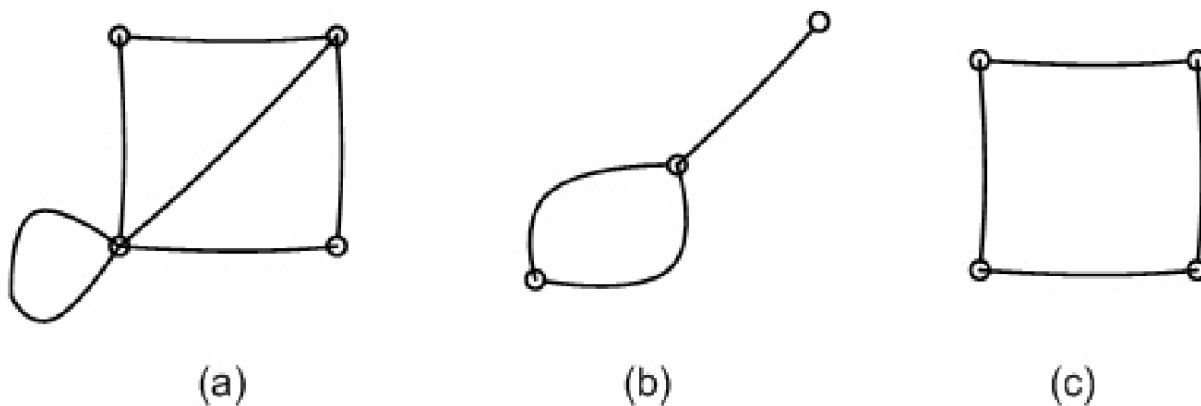
有向图的出度（out-degree）表示为 $d^+(V)$

有向图中用|A|表示边数量，那么入度和出度有如下公式

$$\sum_i d^-(V) = \sum_i d^+(V) = |A|$$

子图 Subgraph

对于这个例子，很显然图c是图a的子图



连通图 connected graph

如果存在从图形 G 的任何一个顶点到任何其他顶点的路径，则图形 G 是连通的。比如树就是一种连通图。

分量 Component

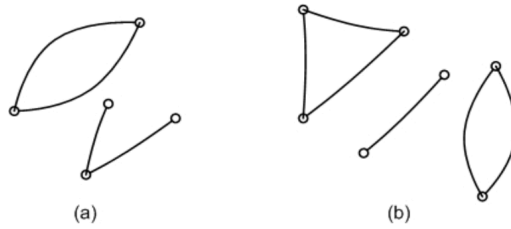
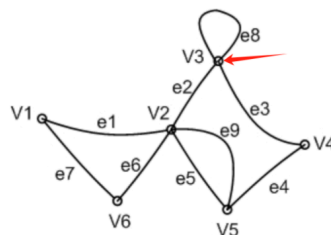


图 (a) 有两个分量，图 (b) 有三个分量

邻接矩阵、邻接表、关联矩阵、关联表

	有向图	无向图
邻接矩阵 + 邻接表	$ \begin{matrix} & a & b & c & d & e \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} & \begin{matrix} a & b \\ b & c \\ c & d \\ d & e \\ e & a \end{matrix} \end{matrix} $	$ \begin{matrix} & a & b & c & d & e \\ \begin{matrix} a \\ b \\ c \\ d \\ e \end{matrix} & \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} & \begin{matrix} a & b & c & d \\ b & c & d & e \\ c & d & e & a \\ d & e & a & b \\ e & a & b & c \end{matrix} \end{matrix} $
关联矩阵 + 关联表	$ \begin{matrix} & a & b & c & d & e \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 0 & -1 \end{pmatrix} & \begin{matrix} 1 & a & b \\ 2 & c & b \\ 3 & b & d \\ 4 & b & e \\ 5 & d & e \\ 6 & c & e \end{matrix} \end{matrix} $	$ \begin{matrix} & a & b & c & d & e \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix} & \begin{matrix} 1 & a & c \\ 2 & a & d \\ 3 & b & c \\ 4 & b & d \\ 5 & c & d \\ 6 & d & e \\ 7 & c & e \end{matrix} \end{matrix} $ labels of edge are edge number

需要注意的是：关联矩阵指的是点和边的接触次数。如果是（五环tu）自环，就是2了

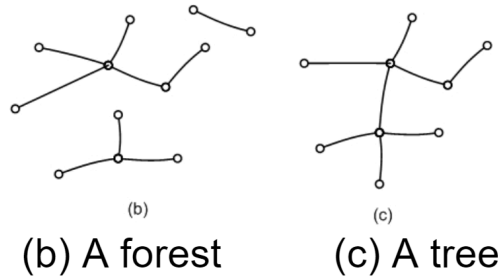


关联矩阵

The incidence matrix for the graph is given by

$$\begin{matrix} & e1 & e2 & e3 & e4 & e5 & e6 & e7 & e8 & e9 \\ \begin{matrix} V1 \\ V2 \\ V3 \\ V4 \\ V5 \\ V6 \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix}
 \end{matrix}$$

树和森林



树的性质

- 若树 T 至少有两个顶点，则它具有以下性质：
- 对于树 T 中的任意顶点 V_i 到任意其他顶点 V_j ，恰好存在一条路径。
- 从树 T 中移除任意一条边后得到的图有两个连通分量，且每个连通分量都是一棵树。
- 边数 $|E|$ 等于顶点数 $|V|$ 减 1。

生成树

如何绘制一棵生成树？

- 选取图 G 的任意一个顶点作为初始的部分树。
- 逐边添加，使得每条新添加的边将一个新顶点连接到部分树上。
- 何时停止？
- 如果图 G 中有 n 个顶点，那么生成树将有 n 个顶点和 $n - 1$ 条边。

最小生成树

采用贪心算法（greedy）来完成，参考INT102的Kruskal和Prim算法。

Dijkstra算法

[查看 INT102的Dijkstra算法笔记](#)

